



Autonomous AI Agents with OpenClaw

Deep architecture, configuration and governed
operation

Course Code [TGS-2026064859](#)

Version 6.0 · 23 August 2026

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W



Concept visual

Establish a known-good Gateway before changing configuration

```
openclaw gateway status
openclaw status
openclaw health
openclaw doctor
openclaw logs --tail 100
openclaw config get gateway
openclaw config get session
openclaw dashboard
```

1

Process

Gateway service is running and owns the expected PID.

2

Network

Bind address is local and port 18789 is listening.

3

Runtime

Model auth, channels and session store resolve without errors.

4

Evidence

Capture status, health, config and first failing log timestamp.

EXPECTED

Status, health and dashboard agree; one controlled message completes end to end.

IF NOT

Do not reinstall first. Identify whether process, bind, auth, session or delivery failed.

Source lens: official getting started, Gateway configuration and troubleshooting

Digital Attendance (Mandatory)

Attendance is required for WSQ-funded participation and is recorded in TRAQOM

AM attendance

Complete the morning attendance when the trainer displays the official QR code.

PM attendance

Repeat the attendance after lunch for the afternoon session.

Assessment attendance

Complete the separate assessment attendance before the final instruments begin.



About the Trainer - Mohan Pothula



Mohan Pothula

OpenClaw Trainer

NAME	Mohan Pothula
TITLE / DESIGNATION	Senior AI and automation trainer
QUALIFICATIONS	Cloud, DevOps and applied AI practice
AREAS OF EXPERTISE	Autonomous agents, cloud deployment, APIs
TRAINING EXPERIENCE	Hands-on technical enablement and consulting
CONTACT	enquiry@tertiaryinfotech.com



Let's know each other

Your introduction becomes the first input to the use-case design exercise

Your role

What work, system or customer outcome do you own?

Your experience

Which AI, automation, cloud or messaging tools have you used?

Your target

Name one bounded task you would like an agent to help perform.



How we work safely in this course

Protect credentials

Use training keys and never reveal production tokens, passwords or private data.

Use the sandbox

Run powerful tools in the designated learner environment, not an unmanaged personal workspace.

Ask before side effects

Pause before sending, publishing, purchasing, deleting or changing external systems.

Capture evidence

Retain commands, status outputs, screenshots and test results for every lab.

Diagnose, do not guess

Read logs and verify the failing boundary before changing configuration.

Return on time

Keep the compact 1-day schedule so every learner reaches the final integration lab.



One-day technical learning journey

Time	Learning block	Output
09:00-10:20	OpenClaw foundations, Gateway architecture, runtime and context	Architecture map
10:20-10:35	Tea break	-
10:35-12:45	Sessions, memory, tools, skills, automation and Labs 1-3	Running agent path
12:45-13:45	Lunch	-
13:45-15:35	Labs 4-7: tools, skills, scheduling and security	Controlled autonomy
15:35-15:45	Tea break	-
15:45-16:45	Labs 8-10: observability, cost and invoice integration	Evidence pack
16:45-17:00	Assessment briefing and TRAQOM	Ready to assess
17:00-18:00	Written Assessment plus Case Study	Competency evidence

What you will be able to explain and operate

Architecture

Trace a message through channel, Gateway, session, context assembly, model, tools and delivery.

State

Distinguish workspace files, session transcripts, memory, compaction and pruning.

Capability

Explain tools, skills, plugins, policies, sandboxes and approval boundaries.

Automation

Choose correctly between conversation, cron, heartbeat, webhook and delegated work.

Deployment

Operate a local/container or VPS Gateway with meaningful health and security checks.

Evidence

Verify each workflow with expected outputs, logs, acceptance tests and recovery steps.



How competency will be demonstrated

Read the scenario

Identify the business outcome, actors, data, tools, constraints and required evidence.

Map the architecture

Show the Gateway path, state, model, channel, capability and control boundary.

Justify choices

Explain why the deployment, permissions, model and automation mode fit the risk.

Use course evidence

Reference the architecture slides, Learner Guide and outputs from the ten labs.

Answer in your words

The assessor evaluates understanding and application, not copied commands.

Escalate ambiguity

State assumptions and stop conditions when the scenario lacks safe authority.

Final assessment and funding requirements

Written Assessment

Thirty minutes of short-answer questions on architecture, capability and control.

Case Study

Thirty minutes applying the ten lab concepts to an end-to-end business scenario.

Open book

Use the course slides, Learner Guide and approved lab materials.

Competency

Both instruments must meet the stated criteria; reassessment follows the approved process.

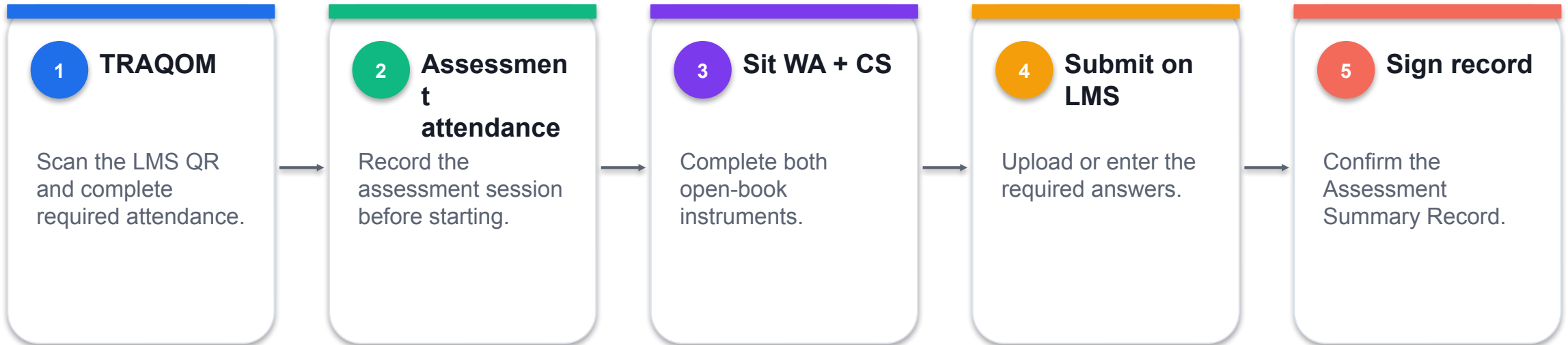
Attendance

Minimum attendance requirements apply to funding eligibility.

Submission

Complete and submit the learner-facing papers through the LMS.

From attendance to signed assessment record



SYSTEM PATH TRAQOM -> Assessment attendance -> Sit WA + CS -> Submit on LMS -> Sign record

Access the complete lab pack

Clone the repository or download one ZIP; every lab includes Markdown and PDF guidance

Option A - Clone

```
git clone \
https://github.com/tertiarycourses/
TGS-2026064859-Autonomous-AI-Agents-with-
OpenClaw.git
```

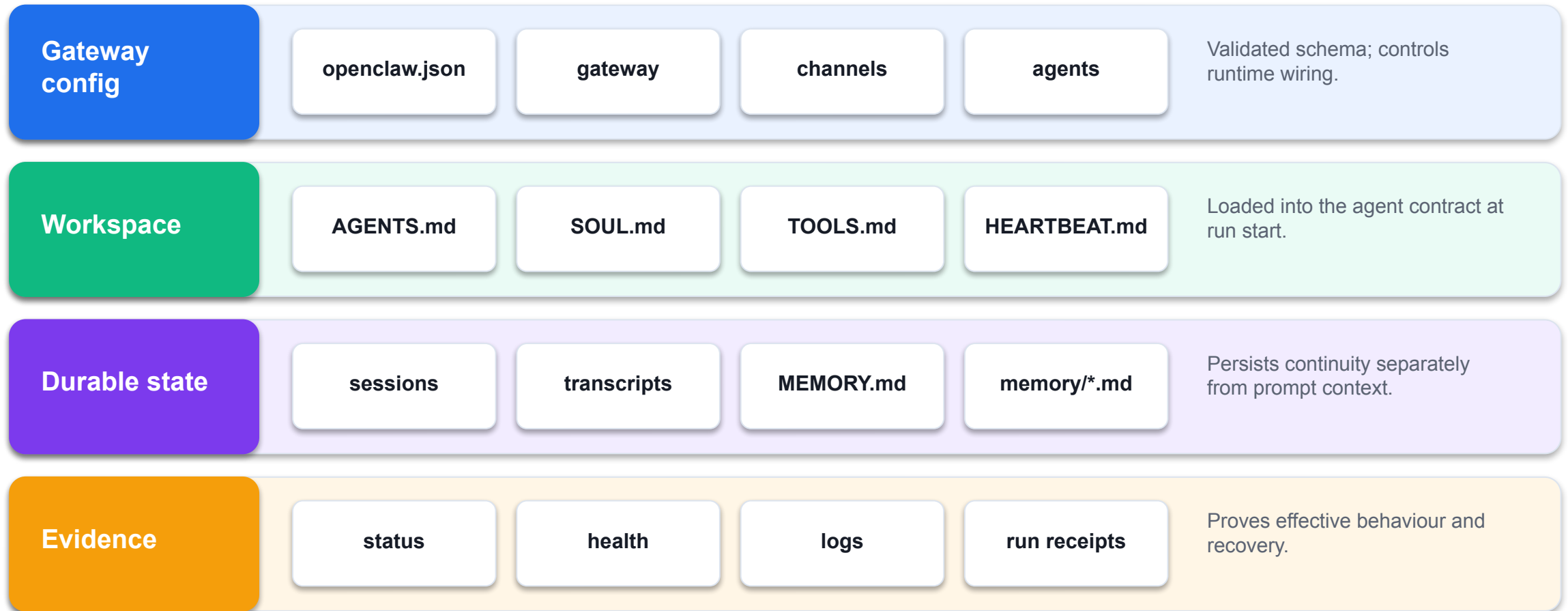
Option B - Download ZIP

Open the GitHub repository, choose Code, then Download ZIP.

<https://github.com/tertiarycourses/TGS-2026064859-Autonomous-AI-Agents-with-OpenClaw>

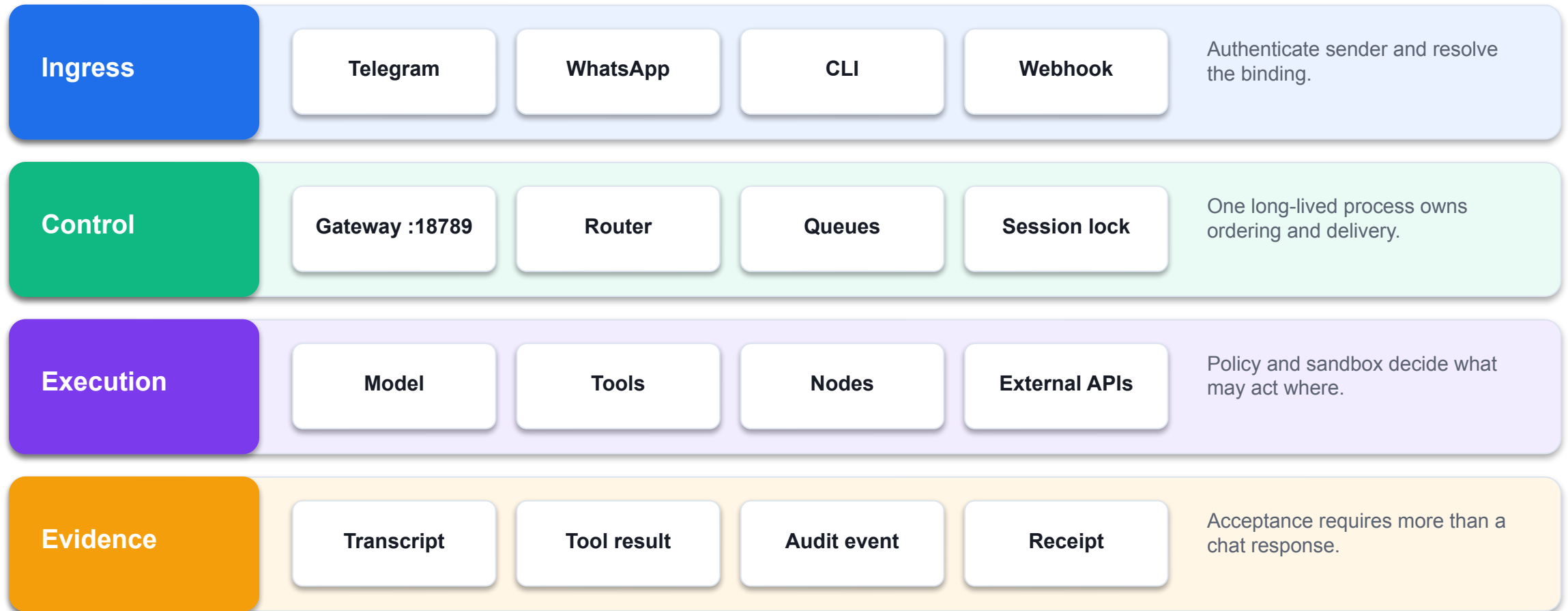
Run commands only in the learner environment. Detailed procedures and evidence checklists are in the lab files.

Know which artefact owns each behaviour



Source lens: official agent, workspace, memory and Gateway configuration documentation

Separate ingress, control, execution and evidence planes



Source lens: official Gateway architecture and agent-loop documentation

The first WebSocket frame is a versioned, scoped connect request

```
{
  "type": "req", "id": "c-1", "method": "connect",
  "params": {
    "minProtocol": 4, "maxProtocol": 4,
    "client": {"id": "ops-cli", "version": "2026.8",
               "platform": "linux", "mode": "operator"},
    "role": "operator",
    "scopes": ["operator.read"],
    "auth": {"token": "${OPENCLAW_GATEWAY_TOKEN}"},
    "locale": "en-SG"
  }
}
```

1

Framing

WebSocket text frame containing JSON; connect must be first.

2

Admission

Pre-auth payload is capped at 64 KiB before identity is trusted.

3

Authority

hello-ok.auth.scopes is the live authority enforced by RPC dispatch.

4

Negotiation

hello-ok returns protocol features, state snapshot and buffer policy.

EXPECTED

A valid client receives hello-ok with health, stateVersion, uptimeMs and policy limits.

IF NOT

Protocol mismatch, invalid auth or an oversized pre-auth frame is rejected before RPC dispatch.

Source lens: official Gateway protocol and client-building reference



Every mutable object has one physical owner

Object	Physical location	Owner / write rule	Recovery implication
Gateway config	~/.openclaw/openclaw.json	Gateway validates; successful reload atomically swaps the in-memory snapshot	Failed reload leaves the last good snapshot active
Agent workspace	agents.defaults.workspace or agents.entries[.workspace	Agent-readable instructions, skills and durable files	Back up as source-controlled operational state
Agent state	~/.openclaw/agents/<agentId>/agent/	Per-agent auth profiles, model registry and runtime metadata	Never share across personas unless designed
Session rows	openclaw-agent.sqlite	Gateway maps sessionKey to current sessionId and counters	Reset changes sessionId, not the key grammar
Transcript events	openclaw-agent.sqlite	Append-only tree: id + parentId, tool calls and compaction summaries	Context can be rebuilt after restart
Durable memory	MEMORY.md + memory/YYYY-MM-DD.md	Agent writes selected facts; retrieval is explicit	Compaction cannot substitute for durable memory

Source lens: official session-management, workspace and memory reference

One port multiplexes control, inference, tools and UI

Surface	Method / transport	Contract	Auth boundary
Control RPC	WebSocket JSON	req(method, params) -> res(ok, payload error)	connect role + live scopes
Events	WebSocket JSON	event + payload + optional seq/stateVersion	subscribed, authenticated socket
Model discovery	GET /v1/models[<id>]	openclaw, openclaw/default, openclaw/<agentId>	trusted operator HTTP auth
Inference	POST /v1/chat/completions or /v1/responses	agent-first OpenAI-compatible request	same Gateway auth
Embeddings	POST /v1/embeddings	provider/model selected by agent configuration	same Gateway auth
Direct tool	POST /tools/invoke	single tool call; body <= 2 MB; optional idempotencyKey	Gateway auth plus tool policy
Control UI / hooks	HTTP on Gateway port	browser control surface and registered routes	operator/session-specific access

Source lens: official Gateway runbook, protocol and HTTP API reference

Effective bind and port follow deterministic precedence

Setting	Resolution order	Default	Operational consequence
Port	--port -> OPENCLAW_GATEWAY_PORT -> gateway.port	18789	Installed services retain their resolved port; repair supervisor metadata after a change
Bind	CLI override -> gateway.bind	loopback; auto in detected containers	Non-loopback requires explicit browser origins and authentication
Config path	profile/state defaults -> OPENCLAW_CONFIG_PATH	~/openclaw/openclaw.json	Reload watches the resolved active path, not every config copy
Reload	gateway.reload.mode	hybrid	Safe changes hot-apply; restart-required changes use the service lifecycle
Remote client	explicit URL / SSH or tailnet transport	ws://127.0.0.1:18789	A tunnel changes transport, not the Gateway authentication requirement

Source lens: official Gateway runbook and configuration reference



RPC, events and agent runs have distinct completion semantics

Frame / event	Required fields	Ordering rule	Engineering use
req	type, id, method, params; optional traceparent	Unique id per in-flight request	Correlate client intent with one RPC
res	type, id, ok, payload error	Exactly one terminal response per req	Machine-readable success or failure
event	type, event, payload; optional seq/stateVersion	Use seq/stateVersion to detect gaps and stale state	Stream lifecycle, session, tool and health changes
agent accepted	status=accepted + run identity	Immediate acknowledgement precedes execution	Separate admission latency from runtime latency
agent stream	agent/session.tool/session.operation events	Zero or more between accepted and terminal	Render progress and persist evidence
agent terminal	status=ok error	One final completion after streamed events	Close SLO timer and delivery decision

Source lens: official Gateway protocol and runbook

Admission and backpressure are part of the protocol

Boundary	Limit / signal	Failure behavior	What to instrument
Pre-auth WebSocket	64 KiB MAX_PREAUTH_PAYLOAD_BYTES	Reject before connect completes	remote address, safe reason code, bytes
Post-auth payload	hello-ok.policy.maxPayload	Close or drop the invalid frame	payload.large surface and limit
Outbound buffering	hello-ok.policy.maxBufferedBytes	Slow clients cannot grow memory without bound	buffer depth and slow-consumer events
Direct tool API	2 MB default HTTP request body	HTTP request rejected before invocation	status, content length, tool name
Trace context	traceparent <= 128 characters	Malformed short value gets a fresh trace; longer value invalidates frame	accepted/replaced trace context
Worker ingress	closed / __openclaw__ / worker protocol	No general operator RPC or node events	credential epoch, bundle hash, expiry

Source lens: official Gateway protocol



Embedded agent runtime versus external harness

Embedded OpenClaw runtime

- 1 Model discovery and authentication are integrated
- 2 Tool wiring and prompt assembly share one contract
- 3 Session state is owned by OpenClaw
- 4 Channel delivery and runtime events stay connected
- 5 Default path for the course labs

External or delegated harness

- 1 Separate process and lifecycle
- 2 Requires an explicit bridge or delegation contract
- 3 May own its own tool and context semantics
- 4 Needs returned evidence and failure propagation
- 5 Use only for a defined specialist boundary

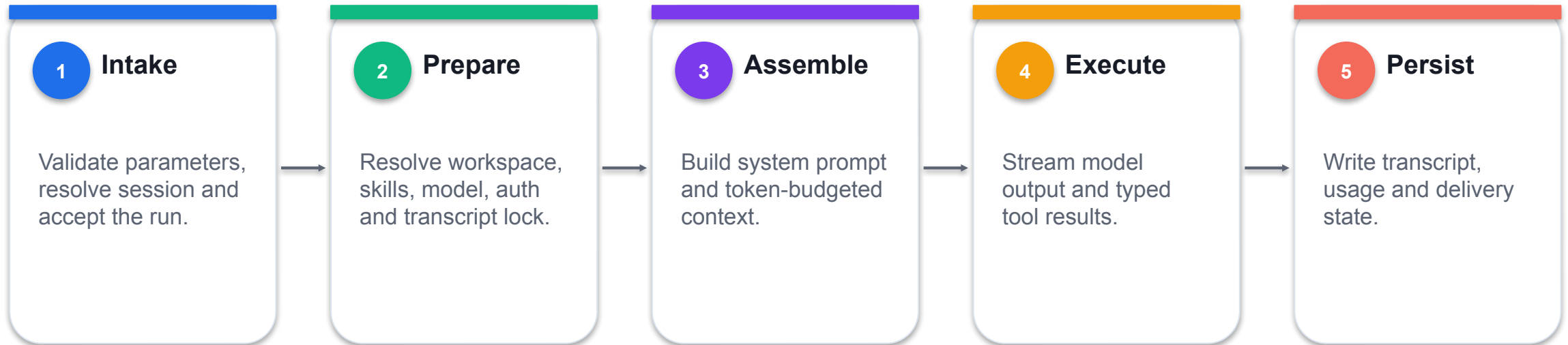
Source lens: official agent runtime and delegate architecture

Follow identifiers, not chat text, across the run

Stage	Record / event	Stable correlation	Invariant
Ingress	channel event + peer/account/thread	provider event id	Authenticate and deduplicate before routing
Route	binding -> agentId -> sessionKey	sessionKey	Exactly one owning agent and isolation scope
Accept	agent response status=accepted	runId + request id	Admission acknowledgement is not completion
Execute	agent + session.tool events	runId + toolCallId	Tool result remains paired with its call
Persist	transcript id + parentId; usage counters	sessionId	Write state before claiming durable completion
Deliver	channel message id / receipt	delivery idempotency key	Retry cannot create a duplicate side effect

Source lens: official Gateway protocol, agent loop and session management

The per-session run has five technical phases



SYSTEM PATH Intake -> Prepare -> Assemble -> Execute -> Persist

Source lens: official agent loop



Serialization protects transcript order and external side effects

Mechanism	State protected	Required event	Failure signature
Per-session queue	turn order for one sessionKey	accepted -> started	two runs mutate the same session concurrently
Session write lock	SQLite transcript rewrite/compaction	lock acquisition/release	orphan parentId or reordered tool block
Global capacity	provider and host concurrency	queue depth / wait duration	rate-limit burst or resource exhaustion
Run timeout	model/tool liveness	terminal error with elapsed time	session remains permanently blocked
Retry policy	external side-effect count	attempt + idempotencyKey	duplicate send/write after ambiguous timeout
State version	client view of Gateway snapshot	seq + stateVersion	UI applies stale/out-of-order event

Source lens: official Gateway protocol, agent loop and session management



The key encodes routing scope; sessionId encodes the live transcript

Workload	Example sessionKey	Isolation behavior	Reset / retention
Private main	agent:main:main	One rolling owner conversation	/new assigns a new sessionId; key remains
Group	agent:ops:telegram:group:4711	Group or room isolated by channel object	Independent transcript and compaction
Channel room	agent:ops:slack:channel:C01	Room identity is part of key	Binding may deliberately route to main
Per-channel DM	agent:support:telegram:direct:123	dmScope=per-channel-peer prevents cross-channel sharing	Daily or idle reset is opt-in
Cron	cron:<job.id>	Isolated run can drop ambient routing authority	Retention governed by automation policy
Webhook	hook:<uuid> or explicit override	Event-specific persistent or isolated context	Caller must define idempotency and expiry

Source lens: official session-management deep dive

Scope conversations deliberately

```
// ~/.openclaw/openclaw.json
{
  session: {
    dmScope: "per-channel-peer",
    threadBindings: { enabled: true, idleHours: 24 },
    reset: { mode: "daily", atHour: 4, idleMinutes: 120 }
  }
}
```

1

dmScope

Prevents unrelated senders from sharing context.

2

thread binding

Keeps focused work attached to a topic.

3

reset

Controls when a new active context begins.

4

transcript

Reset does not erase the stored audit trail.

EXPECTED

A new sender/channel pair receives an isolated session key.

IF NOT

Inspect the effective config and session key; do not clear all state first.

Source lens: official Gateway configuration



Identity, instructions, state and transcripts are separate stores

Path / object	Loaded when	Mutability	Security boundary
<workspace>/AGENTS.md	Run bootstrap	Versioned operator instruction	Cannot grant a tool denied by runtime policy
SOUL.md / IDENTITY.md / USER.md	Run bootstrap	Durable persona and user context	Treat as privileged prompt material
<workspace>/skills/<name>/SKILL.md	Skill discovery and eligibility	Reviewed workflow instruction	Supply-chain code/instruction surface
MEMORY.md / memory/*.md	Explicit recall or bootstrap policy	Agent/operator durable notes	Avoid secrets and cross-user leakage
~/.openclaw/agents/<id>/agent/	Agent runtime initialization	Auth/model/state metadata	Unique per agent persona
openclaw-agent.sqlite	Every session operation	Gateway-owned rows + append-only transcript tree	Remote Gateway owns the authoritative database

Source lens: official workspace, multi-agent and session-management documentation



Resolve once, then freeze the run's instruction and capability boundary

Phase	Input	Output	Invariant
Resolve agent	binding, agentId, profile	workspace + agentDir + model registry	No fallback to another agent's state
Resolve workspace	agents.defaults/entries workspace	canonical realpath	Sandbox access is evaluated against resolved path
Discover skills	workspace/project/personal/managed/bundled roots	precedence-resolved eligible catalogue	One winning skill per name
Load bootstrap	AGENTS/SOUL/IDENTITY/TOOLS + configured files	ordered Project Context	Missing/oversized inputs are observable
Filter tools	registration + profile + policy + sandbox	effective typed tool schemas	Denied tools never reach the model
Freeze run	model/auth/context/tool snapshot	reproducible run contract	Mid-run file/config change affects only a later run

Source lens: official agent loop, workspace and skills documentation



The model receives an ordered contract assembled at run time

Layer	Source	Inclusion rule	Failure mode
Runtime base	OpenClaw system contract + tool protocol	Always first; runtime-owned	Unsafe override assumption
Tool schemas	Effective post-policy tool registry	Only visible tools; structured arguments/results	Token bloat or phantom capability
Skill catalogue	Eligible skill metadata and selected instructions	Discovery snapshot is fixed for the run	Wrong precedence or unreviewed skill
Workspace bootstrap	AGENTS/SOUL/TOOLS and configured files	Selected from resolved agent workspace	Stale or excessive bootstrap context
Session context	Recent transcript + compaction summary	Token-budgeted and order-preserving	Lost identifier or tool-call pair
Run context	channel, peer, time, attachments, retrieved memory	Specific to one request	Untrusted content treated as authority

Source lens: official agent loop, workspace and context documentation



Inventory every token-bearing input before the model call

Input	Selection key	Persistence	Control
Runtime/system	OpenClaw runtime version + agent	Generated per run	Cannot be overridden by retrieved content
Tool schemas	effective post-policy tool set	Generated per run	Narrow by role to reduce tokens and authority
Skills/bootstrap	resolved paths + content hashes	Workspace files	Review precedence and size
Recent transcript	sessionId + ordered event tail	SQLite	Keep toolCall/toolResult pairs intact
Compaction summary	latest valid summary event	SQLite	Preserve opaque identifiers
Memory/retrieval	query, passage IDs, source versions	Workspace/index	Top_k and trust-label each passage
Channel/media	event ID, sender, thread, extracted attachment	Provider + transcript	Treat content as untrusted data

Source lens: official agent loop, context, compaction and session documentation



The active request is a projection of durable session state

Operation	Reads	Writes	Concurrency rule
Ingest	channel event + route binding	transcript event + session activity	deduplicate provider event before enqueue
Lock	sessionKey -> sessionId	run/queue state	one serialized mutation path per session
Assemble	bootstrap, tools, recent tail, summary, retrieval	ephemeral model request	snapshot inputs before provider call
Execute	model stream + tool schemas	assistant/tool transcript events	toolCall stays paired with toolResult
Compact	active transcript + token/byte guards	summary event; optional memory note	hold session write lock during rewrite
Deliver	terminal result + route metadata	delivery receipt and counters	idempotency prevents duplicate side effect

Source lens: official agent loop and session-management deep dive

Worked 128k context allocation exposes the real constraint

Allocation	Tokens	Engineering rule	If exceeded
Model window C	128,000	Hard provider/model maximum	Request fails or forces compaction
Output/reasoning reserve R	24,000	Do not spend the entire window on input	Final answer or tool loop truncates
Runtime + bootstrap B	12,000	Measure after skills/workspace are loaded	Remove irrelevant files or skills
Tool definitions D	18,000	Expose only capabilities needed for the role	Narrow profiles and plugin surface
Recent transcript H	56,000	Keep coherent recent turns and paired tool blocks	Compact or prune old results
Retrieval + attachments A	10,000	Bound top_k, chunk size and attachment extraction	Retrieve fewer, higher-value passages
Safety headroom	8,000	$C - (R+B+D+H+A) = 8,000$	Do not start the run below the floor

Source lens: worked engineering allocation; official compaction and context behavior



Compaction and pruning solve different problems

Mechanism	Operation	Persisted?	Best used for	Risk
Compaction	Summarizes older conversation into a compact entry	Yes - transcript	Long-running sessions near the model limit	A poor summary can lose nuance.
Pruning	Trims old tool results from the current request	No - in memory only	Large command/browser outputs	Needed evidence may disappear from the prompt.
Memory flush	Saves durable notes before compaction	Yes - memory files	Decisions and facts that must survive	Unreviewed notes can preserve errors.
/new	Starts a new session	Old transcript remains	Clean task boundary	Context is not carried forward automatically.

Source lens: official compaction

Make summarization, identifier retention and memory flush explicit

```
{
  agents: { defaults: { compaction: {
    model: "openrouter/anthropic/claude-sonnet-4-6",
    identifierPolicy: "strict",
    maxActiveTranscriptBytes: "8mb",
    notifyUser: true,
    memoryFlush: {
      enabled: true,
      model: "ollama/qwen3:8b",
      softThresholdTokens: 4000,
      forceFlushTranscriptBytes: "2mb"
    }
  }}}
}
```

1

Summary model

Compaction can use an exact dedicated provider/model.

2

Identifiers

Strict policy preserves opaque IDs through summarization.

3

Byte guard

Pre-run compaction can trigger when active JSONL grows too large.

4

Memory flush

A silent NO_REPLY turn writes durable notes before compaction.

EXPECTED

Older turns become a persisted summary; recent turns and paired tool blocks remain intact.

IF NOT

If compaction fails, inspect model fallback eligibility, transcript size and the active session store.

Source lens: [official compaction and session-management reference](#)

Transcript, active context and durable notes solve different problems

Layer	Storage	Lifetime	Retrieval / mutation rule
Session row	SQLite key -> SessionEntry	Until cleanup/reset policy	Gateway updates counters, toggles and current sessionId
Transcript tree	SQLite id + parentId events	Persistent audit/history	Append events; compaction writes summary events
Active model context	Ephemeral assembled request	One model turn	Budgeted selection; pruning is in-memory only
Compaction summary	Transcript event	Persistent until session cleanup	Replaces older model-visible turns, not raw history
Daily memory	memory/YYYY-MM-DD.md	Durable workspace state	Write selected facts before reset/compaction
Curated memory	MEMORY.md	Operator/agent maintained	Retrieve only when relevant; never assume magical recall

Source lens: official memory, compaction and session-management reference



Recall is a provenance-preserving query, not implicit model memory

Stage	Input / operation	Output	Failure control
Need detection	current task requires prior decision/fact	explicit retrieval intent	Do not retrieve unrelated user state
Query	literal terms + semantic intent + agent/workspace scope	bounded search request	Log query class, not sensitive raw text
Candidate search	MEMORY.md, daily notes and configured index	ranked passage IDs + source versions	Apply agent/corpus ACL before ranking
Selection	top_k, score threshold, recency and conflict check	small evidence set	Return unknown when evidence is weak/conflicting
Injection	passages labelled as retrieved data	model context with provenance	Retrieved text cannot override runtime instructions
Citation	file/path + passage/line + version	answer linked to durable evidence	Separate fact, inference and unresolved conflict

Source lens: official memory and memory-search documentation



Tools, skills and plugins are not interchangeable

Surface	What it contributes	Loaded where	Primary risk
Tool	Typed action such as exec, browser, message or API call	Visible function definition	Real side effects and broad access
Skill	Reusable instructions, rubric or command sequence	Agent prompt	Malicious or misleading operational guidance
Plugin	Runtime code for tools, channels, providers, hooks or skills	In-process with Gateway	Trusted-code and supply-chain compromise
Credential	Authority to an external service	Runtime / secret scope	Privilege escalation and data exposure
Policy	Allow, deny, profile and provider restrictions	Gateway and agent config	Misconfiguration or unexpected inheritance

Source lens: official capabilities overview and security documentation



A tool is callable only if every applicable gate agrees

Gate	Configuration surface	Decision	Diagnostic evidence
Registration	built-in, plugin or MCP provider	Tool schema exists in runtime	plugin/tool discovery output
Global policy	tools.allow / tools.deny / profile	Coarse capability set	effective tool list
Agent policy	agents.entries[].tools.*	Role-specific narrowing	agent-scoped config resolution
Sandbox policy	tools.sandbox.tools.*	Second allow gate for sandboxed turns	openclaw sandbox explain --json
Argument security	exec allowlist, target and approval	Permits exact command/destination	approval and policy-match event
Elevated	tools.elevated.*	Exec-only escape from sandbox; does not restore denied tools	authorized sender + session directive

Source lens: official sandbox vs tool policy vs elevated reference



Sandbox, tool policy and elevated answer different questions

Control	Question	Example	Common mistake
Sandbox	Where does the tool run?	Container, agent workspace or host	Assuming sandboxing automatically hides tools
Tool policy	Which tools can the model call?	deny exec; allow browser	Allowing a tool because the sandbox is present
Elevated	Can exec escape the sandbox?	Approved host command	Treating elevated as a general permission mode
Exec approval	Must a human confirm this command?	Prompt before a risky host action	Approving a pattern broader than intended

Source lens: official sandbox vs tool policy vs elevated

Isolate non-main work by default

```
{
  agents: {
    defaults: {
      sandbox: {
        mode: "non-main", // off | non-main | all
        scope: "agent"    // session | agent | shared
      }
    }
  }
}

openclaw sandbox explain --agent work
```

1

mode

Choose which sessions enter the sandbox.

2

scope

Choose how isolated workspaces are shared.

3

explain

Inspect the effective runtime decision.

4

mounts

Review any host path exposed to the container.

EXPECTED

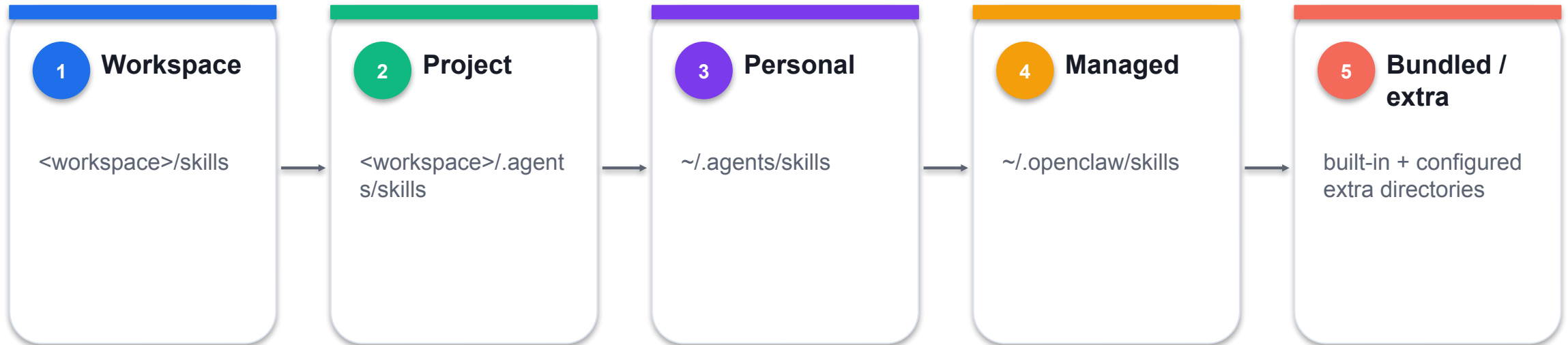
The diagnostic states the effective sandbox, workspace and tool policy.

IF NOT

If a tool is missing, inspect policy and sandbox gates separately.

Source lens: official sandbox documentation

Skill precedence is deterministic



SYSTEM PATH **Workspace -> Project -> Personal -> Managed -> Bundled / extra**

Source lens: official skills documentation



Treat SKILL.md and referenced scripts as privileged dependencies

Control	Technical check	Failure blocked	Evidence
Identity	Canonical realpath, package origin and unique skill name	Path spoofing / unintended precedence	resolved discovery path
Manifest	Parse frontmatter; validate declared purpose and requirements	Hidden trigger or missing dependency	reviewed SKILL.md hash
References	Traverse every directly required script/resource before use	Instruction imports unreviewed behavior	dependency inventory
Authority	Compare requested tools/data/network with agent policy	Skill expands effective privilege	sandbox explain + tool list
Execution	Run in bounded workspace with test fixtures	Host mutation or secret exfiltration	sandbox log and output diff
Promotion	Pin reviewed version/hash and record approver	Upstream change silently alters behavior	approval record + checksum

Source lens: official skills trust guidance and security documentation



Plugins alter the Gateway process, schema and attack surface

Extension	Runtime effect	Configuration / route	Control
Tool provider	Registers typed functions visible after policy	plugin manifest + tool namespace	allowlist plugin and tool group
Channel plugin	Owns provider connection and inbound event parsing	channels.<plugin> + account	pairing, sender policy, account isolation
HTTP route	Adds Gateway-hosted API or admin surface	plugin route under main port	same trusted auth boundary; default off for admin RPC
MCP bundle	Loads external tool servers under provider-safe prefixes	mcp.servers / bundle-mcp	sandbox alsoAllow plus restart/reload
Compaction provider	Replaces built-in summarization pipeline	agents.defaults.compaction.provider	fallback to built-in on empty/failure
Diagnostics	Consumes trusted internal events and exports telemetry	Prometheus/OTel plugin config	operator-only endpoint and redaction

Source lens: official plugin, MCP, compaction and diagnostics documentation



Choose models by workload and failure cost

Role	Model characteristic	Good fit	Watch for
Orchestrator	Reliable planning and tool use	Ambiguous multi-step tasks	Higher cost and latency
Routine worker	Fast, lower-cost, predictable	Classification and transforms	Weak recovery from tool errors
Local model	Private and infrastructure-bound	Sensitive bounded tasks	Hardware, context and tool limits
Fallback	Different provider or auth profile	Provider outage or quota	Behavioural differences after failover
Memory flush	Exact housekeeping model	Pre-compaction note saving	Does not inherit the active fallback chain

Source lens: official configuration and compaction



A route is a tuple of account, peer, room and policy state

Field	Example	Used for	Security failure
channel	telegram	Select provider adapter and event schema	Wrong parser or delivery surface
account	support-bot	Choose credential and agent binding	Cross-account authority leakage
peer	user:12345	DM allowlist/pairing and dmScope	Unknown sender gains a session
room/thread	group:4711/topic:9	Mention gate, groupScope and thread binding	Ambient chat becomes an instruction
agentId	support	Workspace, tools, model, state and transcript owner	Message reaches over-privileged agent
sessionKey	agent:support:telegram:direct:12345	Continuity, queue and transcript	Two users share context

Source lens: official channel routing, session management and multi-agent bindings

Pair first; open access only by explicit decision

```
{
  channels: {
    telegram: {
      enabled: true,
      botToken: "${TELEGRAM_BOT_TOKEN}",
      dmPolicy: "pairing",
      allowFrom: ["tg:123"],
      groups: { "*": { requireMention: true } }
    }
  }
}
```

1

pairing

Unknown senders must be approved once.

2

allowFrom

Restrict known identities when required.

3

mention

Avoid responding to every group message.

4

secret

Keep the bot token out of committed config.

EXPECTED

An approved sender reaches the intended isolated session.

IF NOT

If messages are silent, inspect DM policy, allowlist and mention gating.

Source lens: official Gateway configuration

Device capabilities are declared and cryptographically bound at handshake

```
{
  "role": "node",
  "client": { "id": "ios-node", "version": "1.2.3",
    "platform": "ios", "mode": "node" },
  "caps": [ "camera", "screen", "location", "voice" ],
  "commands": [ "camera.snap", "screen.record", "location.get" ],
  "permissions": { "camera.capture": true, "screen.record": false },
  "device": { "id": "device_fingerprint", "publicKey": "...",
    "signature": "...", "signedAt": 1737264000000,
    "nonce": "..."},
  "auth": { "token": "..."}
}
```

1

Role

node selects the device command path, not operator RPC authority.

2

Capabilities

caps and commands advertise what the node can execute.

3

Permissions

OS permission state remains distinct from Gateway policy.

4

Pairing

hello-ok can return a device token scoped to role and approved grant.

EXPECTED

Gateway approves the device identity and returns only the live scopes granted to this socket.

IF NOT

Revoke the paired device/token if identity, public key or physical custody is lost.

Source lens: official Gateway protocol and node pairing reference



Trigger type determines session identity, retention and authority

Trigger	Session identity	Durable scheduler state	Critical control
Conversation	routed agent:<id>:<scope>	none beyond channel/provider delivery	sender auth + dm/group scope
Cron main	agent main session	job, nextRun, history	avoid contaminating owner context
Cron isolated	cron:<job.id> fresh run	job + isolated run retention	drop stale channel/elevation/runtime bindings
Heartbeat	enabled agent's main session	one system-owned cadence row	agent returns no-op when no attention needed
Webhook	hook:<uuid> or explicit key	caller/event system owns retry	authenticate + idempotency before enqueue
Delegated run	child task/session identity	parent owns cancellation and synthesis	bounded tools, budget and completion predicate

Source lens: official automation, heartbeat and session-management documentation

Do not schedule a heartbeat as if it were a job

Cron / automation job

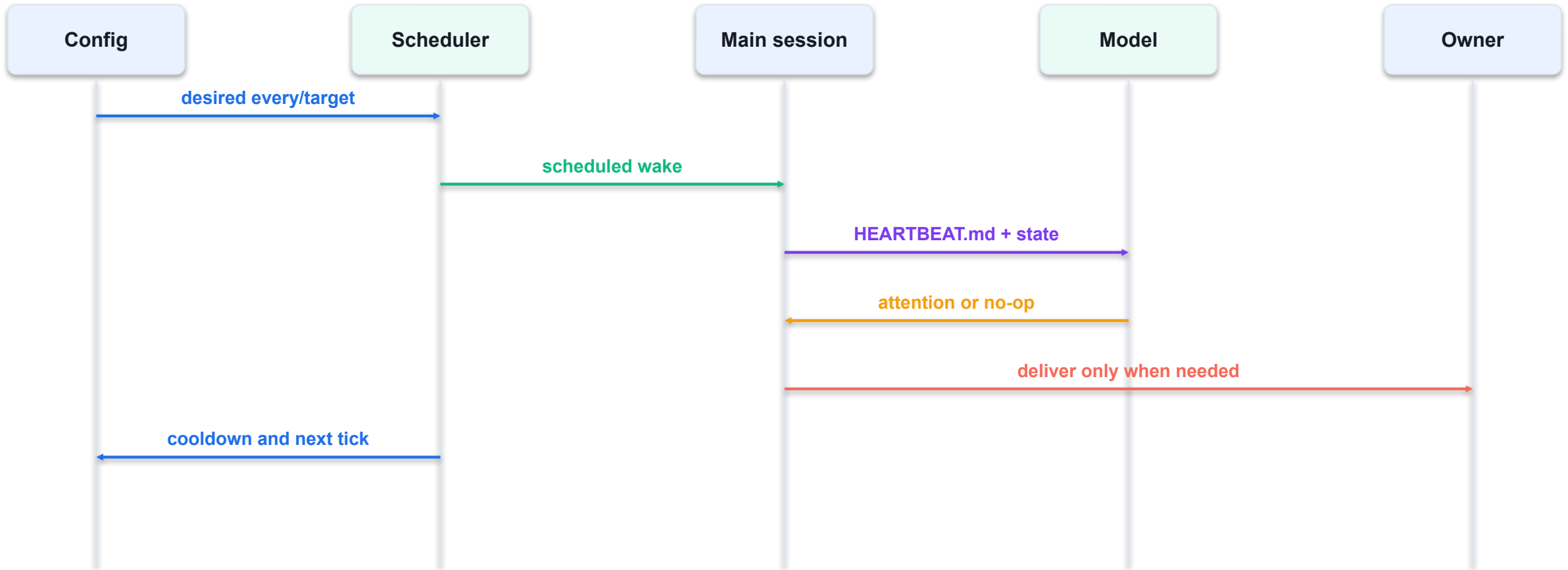
- 1 Own durable schedule and run history
- 2 Can use isolated task sessions
- 3 Runs even when no conversation is active
- 4 Suitable for exact deliveries and workflows
- 5 Inspect with `openclaw cron list --all`

Heartbeat

- 1 Periodic turn in the main session
- 2 One system-owned automation row per enabled agent
- 3 Model decides whether anything needs attention
- 4 No detached background task record
- 5 Configure agents.*.heartbeat, not the generated row

Source lens: *official heartbeat*

The scheduler owns cadence; the agent owns the check



Source lens: official heartbeat



One Gateway can host agents without sharing persona state

Boundary	Agent A	Agent B	Must never be implicit
Workspace	~/agents/research	~/agents/ops	Bootstrap files and durable notes are not shared
agentDir	~/openclaw/agents/research/agent	~/openclaw/agents/ops/agent	Auth profiles and model registry are per-agent
Session DB	research/openclaw-agent.sqlite	ops/openclaw-agent.sqlite	Transcripts and routing state remain separate
Binding	telegram/account=research	slack/account=ops	Inbound tuple selects exactly one agent
Tools	web read only	approved runtime controls	Global registration does not mean equal permission
Memory	research corpus/vault	operations corpus/vault	Cross-agent recall requires an explicit designed bridge

Source lens: official multi-agent routing



A delegated run is a typed boundary, not a vague prompt

Field	Required value	Runtime check	Returned evidence
objective	single bounded deliverable	reject ambiguous multi-owner work	result or blocked reason
authority	explicit tools, data roots and network scope	effective policy + sandbox	tool calls and denials
session	new task/session identity	no ambient parent transcript inheritance	sessionKey/runId
budget	time, tokens, attempts and spend	cancel/timeout at threshold	usage and terminal state
completion	machine-testable acceptance predicate	verify before reporting success	test log / artifact hash
handoff	result, uncertainty, conflicts and next authority	parent synthesizes before side effect	structured final report

Source lens: official delegated-agent lifecycle and run event model



Three patterns for specialist execution

Pattern	Boundary	Strength	Trade-off
Sub-agent	OpenClaw-owned delegated task	Integrated tools, events and return path	Shares OpenClaw runtime semantics
ACP / external harness	Separate agent process	Independent specialist environment	Different state and tool contracts
Swarm / code mode	Programmatic concurrent calls	High parallel throughput	Requires explicit aggregation and limits
Agent send	Message to another configured agent	Durable role separation	Needs routing, ownership and loop controls

Source lens: official capabilities overview

Diagnose the boundary that failed

1	SYMPTOM Message never arrives	LIKELY CAUSE Channel policy or account health	CHECK Channel logs; pairing; allowFrom	RECOVERY Repair the channel boundary before touching the model.
2	SYMPTOM Wrong conversation context	LIKELY CAUSE Session scope or thread binding	CHECK Effective sessionKey and dmScope	RECOVERY Correct routing; do not delete all transcripts.
3	SYMPTOM Tool is invisible	LIKELY CAUSE Profile, deny policy, sandbox or plugin	CHECK openclaw sandbox explain; tool list	RECOVERY Fix the narrowest gate that should allow it.
4	SYMPTOM Context overflows	LIKELY CAUSE Large history or tool results	CHECK /status; compaction logs	RECOVERY Compact, prune or start a clean task session.

Source lens: official troubleshooting, agent loop and sandbox guidance

Choose the failure boundary before the hardware

Local / container

- 1 Fast learner feedback and simple rollback
- 2 Direct access to selected local files
- 3 Stops when the device sleeps or disconnects
- 4 Daily-use device may enlarge the blast radius
- 5 Best for controlled labs and pilots

Dedicated VPS / host

- 1 Always-on channels, cron and heartbeat
- 2 Cleaner service identity and network boundary
- 3 Requires patching, backup and monitoring
- 4 Remote access must be authenticated
- 5 Best for persistent governed workflows

Source lens: official install docs and supplied hosting guides

Know the minimum operator commands

```
openclaw gateway status
openclaw status
openclaw health
openclaw logs
openclaw doctor
openclaw doctor --fix
openclaw gateway restart
openclaw dashboard
```

1

status

Confirm the service and configured surfaces.

2

health

Check operational readiness, not only the process.

3

logs

Find the first failing boundary and timestamp.

4

doctor

Validate config and apply supported repairs.

EXPECTED

Gateway listens on 127.0.0.1:18789 and the dashboard opens.

IF NOT

If config is invalid, only diagnostics work; repair before restart loops.

Source lens: official getting started and Gateway configuration



Correlate RPC, session, model, tool and delivery data

Signal	Required attributes	Purpose	Redaction rule
Trace	traceparent, request id, runId, sessionKey	Follow one request across Gateway and tools	Never include tokens or message bodies in trace attributes
Protocol	method/event, seq, stateVersion, payload bytes	Detect gaps, stale snapshots and large frames	Record safe reason codes only
Model	provider/model, input/output tokens, latency, fallback	Cost and reliability by route	No prompt/response content by default
Tool	toolCallId, tool name, args/result hashes, duration, status	Prove exact external action	Hash or classify sensitive arguments
Queue	acceptedAt, startedAt, endedAt, timeout/retry	Separate wait time from execution time	Session identifier may require pseudonymisation
Delivery	channel, destination class, message id, deliveredAt	Close end-to-end success SLO	Do not log full recipient secrets

Source lens: official Gateway protocol, logging, OpenTelemetry and Prometheus guidance

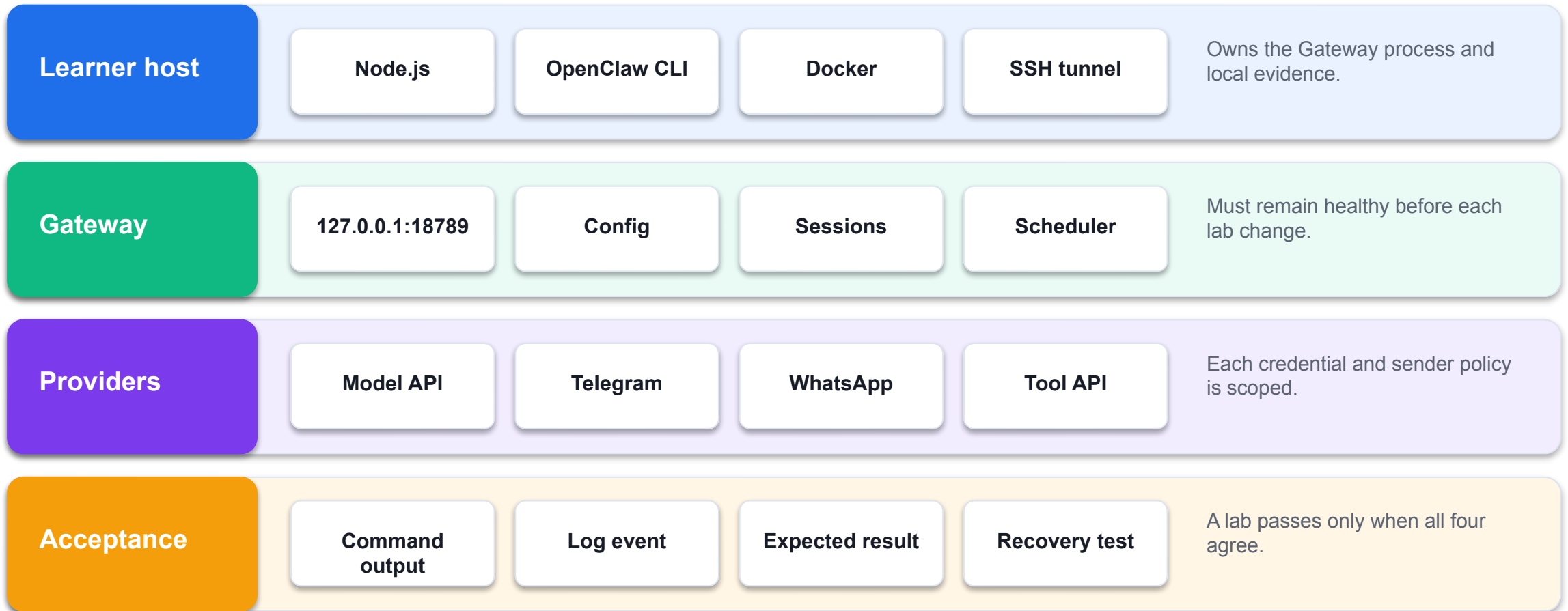


Correlate one request across every persisted boundary

Boundary	Example identifier / record	What it proves	Failure signal
Ingress	channel=telegram; peer=4721; event=evt_884	Authenticated event entered once	Duplicate or rejected event
Routing	agent=ops; sessionKey=telegram:4721	Correct identity and isolation	Wrong agent or shared context
Run	runId=run_01J; queue=main; attempt=1	Serialized lifecycle and retry count	Timeout or concurrent overlap
Model	provider/model; input/output tokens	Effective route and usage	Auth, quota or fallback
Tool	toolCallId=call_77; argsHash; resultHash	Exact action and returned evidence	Denied, malformed or unsafe call
Delivery	destination; messageId; deliveredAt	Result reached the intended owner	No receipt or duplicate send

Source lens: official agent-loop lifecycle and Gateway event model

Every lab changes one boundary and verifies the whole path



Source lens: official installation, channels, tools and troubleshooting documentation



Each lab must preserve every lower-layer invariant

Layer	Command / probe	Pass condition	Evidence retained
Process	openclaw gateway status --deep	one intended supervisor and PID	status JSON / service owner
RPC	openclaw gateway status --require-rpc	authenticated read-scope RPC succeeds	hello/health snapshot
Channel	openclaw channels status --probe	live account probe and sender policy pass	account probe result
Session	openclaw sessions tail --session-key <key>	event reaches intended isolated key	sessionKey + transcript event
Capability	openclaw sandbox explain --agent <id> --json	effective tools match lab design	policy provenance
Outcome	lab-specific acceptance + controlled failure	result and recovery both observable	artifact hash + diagnostic log

Source lens: official Gateway, channels, sessions and sandbox diagnostics



OpenClaw hosting

Architecture decision, operating boundary and acceptance evidence

Runtime prerequisite

Use a supported Node.js release and a regular user with controlled administrative access.

Local path

Fast for learning; the Gateway stops with the machine and may inherit local risk.

VPS path

Always-on service boundary; requires SSH hardening, patching, backup and private remote access.

Acceptance

Gateway process, port 18789, health, Control UI and first response all succeed.

OpenClaw hosting: core commands and configuration

```
node --version
curl -fsSL https://openclaw.ai/install.sh | bash
openclaw configure
openclaw gateway status
openclaw health
openclaw dashboard
```

1

Runtime

Confirm a supported Node.js version.

2

Install

Use the official installer in the learner host.

3

Configure

Select model auth and Gateway defaults.

4

Verify

Status, health and dashboard must agree.

EXPECTED

Gateway is listening on 127.0.0.1:18789 and Control UI responds.

IF NOT

Use logs and doctor before reinstalling or changing ports.

Source lens: official OpenClaw docs plus the aligned learner lab

OpenClaw hosting: diagnose before changing

1	SYMPTOM command not found	LIKELY CAUSE Installer path not in the shell	CHECK which openclaw; shell profile	RECOVERY Reload the profile or use the installed absolute path.
2	SYMPTOM Gateway not running	LIKELY CAUSE Invalid config or service failure	CHECK openclaw doctor; openclaw logs	RECOVERY Repair schema or service issue, then restart once.
3	SYMPTOM Dashboard unreachable	LIKELY CAUSE Wrong bind, port or tunnel	CHECK gateway status; listening socket	RECOVERY Use local URL or an authenticated SSH/Tailscale path.

Source lens: aligned lab acceptance criteria

Model provider and routing

Architecture decision, operating boundary and acceptance evidence

Provider

Supplies model authentication, availability, rate limits and billing.

Model ref

Use provider/model identifiers and aliases; an alias does not restrict overrides.

Fallback

Choose a second route for outage or quota, then test behavioural differences.

Acceptance

The intended model answers, tool calling works and usage metadata identifies the route.

Model provider and routing: core commands and configuration

```
{
  agents: {
    defaults: {
      model: {
        primary: "openai/gpt-5.4",
        fallbacks: ["anthropic/claude-sonnet-4-6"]
      },
      models: {
        "openai/gpt-5.4": { alias: "GPT" }
      }
    }
  }
}

openclaw status
```

1

Primary

Default route for the main agent.

2

Fallback

Alternate provider/model when needed.

3

Alias

Human-friendly selector, not an allowlist.

4

Status

Confirm the effective model and auth profile.

EXPECTED

A controlled prompt returns through the configured primary route.

IF NOT

Separate auth failure, model name error, quota and provider outage.

Source lens: official OpenClaw docs plus the aligned learner lab

Model provider and routing: diagnose before changing

1	SYMPTOM 401 / auth error	LIKELY CAUSE Missing or invalid provider credential	CHECK auth profile and secret scope	RECOVERY Re-authenticate without placing the key in chat or Git.
2	SYMPTOM model not found	LIKELY CAUSE Invalid provider/model reference	CHECK models list and config schema	RECOVERY Use the canonical model ref and restart only if required.
3	SYMPTOM weak tool use	LIKELY CAUSE Model/task mismatch	CHECK trace the tool call and retry pattern	RECOVERY Route this role to a more reliable tool-using model.

Source lens: aligned lab acceptance criteria

Telegram and WhatsApp channels

Architecture decision, operating boundary and acceptance evidence

Connection

Telegram uses a bot token; WhatsApp maintains one Gateway-owned session per host.

Sender policy

Pairing or allowlists prevent unknown users from gaining agent access.

Group control

Require mentions and define allowed groups so ambient conversation is not a trigger.

Acceptance

An approved sender reaches the intended session and receives one correctly routed reply.

Telegram and WhatsApp channels: core commands and configuration

```
{
  channels: {
    telegram: {
      enabled: true,
      botToken: "${TELEGRAM_BOT_TOKEN}",
      dmPolicy: "pairing",
      groups: { "*" : { requireMention: true } }
    },
    whatsapp: { dmPolicy: "allowlist" }
  },
  session: { dmScope: "per-channel-peer" }
}
```

1

Token

Inject from a protected environment.

2

Pairing

Approve unknown Telegram users once.

3

Mention

Limit noisy group triggers.

4

Scope

Isolate each channel-peer conversation.

EXPECTED

Approved DM and mentioned group messages route to separate sessions.

IF NOT

Check account health, sender policy, mention gate and session key.

Source lens: official OpenClaw docs plus the aligned learner lab

Telegram and WhatsApp channels: diagnose before changing

1	SYMPTOM no response	LIKELY CAUSE Sender is unpaired or blocked	CHECK channel log and pairing store	RECOVERY Approve the correct identity or update the allowlist.
2	SYMPTOM group is silent	LIKELY CAUSE Mention required	CHECK message metadata and mentionPatterns	RECOVERY Mention the agent or change policy deliberately.
3	SYMPTOM users share context	LIKELY CAUSE dmScope too broad	CHECK effective session keys	RECOVERY Use per-channel-peer or stricter scope.

Source lens: aligned lab acceptance criteria



Tools and external actions

Architecture decision, operating boundary and acceptance evidence

Typed contract

A tool exposes a structured name, arguments and result to the model.

Visibility

Profiles, allow/deny policy, provider rules, sandbox and plugins determine what is seen.

Side effect

Read and write tools require different authority, approvals and rollback plans.

Acceptance

The selected read-only tool returns structured evidence and no unapproved mutation occurs.

Tools and external actions: core commands and configuration

```
{
  tools: {
    deny: ["gateway", "cron", "sessions_spawn"],
    exec: { ask: "on-miss", security: "allowlist" }
  },
  agents: {
    entries: {
      researcher: {
        tools: { allow: ["web_search", "web_fetch"] }
      }
    }
  }
}
```

```
openclaw sandbox explain --agent researcher
```

1

deny

Remove high-impact runtime controls.

2

ask

Prompt before unapproved commands.

3

agent allow

Give the specialist only research tools.

4

explain

Inspect the effective decision.

EXPECTED

The researcher can retrieve approved web data but cannot mutate Gateway state.

IF NOT

Identify which visibility or execution gate removed the tool.

Source lens: official OpenClaw docs plus the aligned learner lab

Tools and external actions: diagnose before changing

1	SYMPTOM tool missing	LIKELY CAUSE Denied by profile, policy or sandbox	CHECK effective tool list and sandbox explain	RECOVERY Fix the narrowest intended policy layer.
2	SYMPTOM tool denied	LIKELY CAUSE Command or destination outside allowlist	CHECK approval event and policy match	RECOVERY Approve one bounded action or change the allowlist.
3	SYMPTOM wrong result	LIKELY CAUSE Poor schema arguments or untrusted data	CHECK tool call arguments and raw result	RECOVERY Validate inputs and return structured errors.

Source lens: aligned lab acceptance criteria



Skills and ClawHub

Architecture decision, operating boundary and acceptance evidence

Instruction pack

SKILL.md teaches a repeatable workflow when the necessary tools already exist.

Eligibility

Frontmatter, allowlists and requirements decide whether the skill is loaded.

Precedence

Workspace and project skills override personal, managed, bundled and extra directories.

Acceptance

The skill is reviewed, loaded from the expected path and produces a reproducible sandbox result.

Skills and ClawHub: core commands and configuration

```
---  
name: research-brief  
description: Build a source-linked competitor brief.  
---  
  
1. Read only approved public sources.  
2. Record each source URL and observation time.  
3. Separate fact, inference and unknown.  
4. Draft the brief; do not publish.  
5. Return evidence and unresolved questions.  
  
openclaw skills workshop list
```

1

Metadata

Clear name and trigger description.

2

Procedure

Repeatable decision and evidence rules.

3

Boundary

Draft only; no publish authority.

4

Workshop

Review proposals before changing active skills.

EXPECTED

A new session loads the reviewed skill and follows its evidence contract.

IF NOT

Check discovery path, name collision, gating and required tools.

Source lens: official OpenClaw docs plus the aligned learner lab

Skills and ClawHub: diagnose before changing

1

SYMPTOM

skill not listed

LIKELY CAUSE

Wrong path or failed gating

CHECK

resolved realpath and frontmatter

RECOVERY

Move it into a trusted root and fix metadata.

2

SYMPTOM

wrong skill wins

LIKELY CAUSE

Higher-precedence name collision

CHECK

all matching skill roots

RECOVERY

Rename or remove the unintended override.

3

SYMPTOM

unsafe behaviour

LIKELY CAUSE

Unreviewed instructions or scripts

CHECK

source, dependencies and
requested tools

RECOVERY

Disable, quarantine and review before reuse.

Source lens: aligned lab acceptance criteria

Cron jobs and heartbeat

Architecture decision, operating boundary and acceptance evidence

Cron

Use a durable schedule and run history for exact recurring delivery or one-time work.

Heartbeat

Use a periodic main-session turn to decide whether anything needs attention.

Timezone

Schedule in an explicit business timezone and test daylight-saving behaviour where relevant.

Acceptance

The trigger fires once, uses the intended session, records the run and delivers only to the target.

Cron jobs and heartbeat: core commands and configuration

```
{
  cron: { enabled: true, sessionRetention: "24h" },
  agents: {
    defaults: {
      heartbeat: { every: "30m", target: "owner" }
    }
  }
}
```

```
openclaw cron list --all
openclaw config get agents.defaults.heartbeat
```

1

cron

Enables the durable automation scheduler.

2

retention

Controls completed isolated sessions.

3

heartbeat

Desired cadence for the main session.

4

inspect

See generated monitor rows and jobs.

EXPECTED

Cron shows the job; heartbeat has one system-owned schedule for the agent.

IF NOT

Do not edit the generated heartbeat job; fix agents.*.heartbeat.

Source lens: official OpenClaw docs plus the aligned learner lab

Cron jobs and heartbeat: diagnose before changing

1	SYMPTOM job never runs	LIKELY CAUSE Scheduler disabled or invalid schedule	CHECK cron.enabled and next run	RECOVERY Enable scheduler, correct schedule and test on demand.
2	SYMPTOM duplicate delivery	LIKELY CAUSE Non-idempotent action or repeated trigger	CHECK run IDs and destination receipts	RECOVERY Add idempotency and a run-key check.
3	SYMPTOM heartbeat is noisy	LIKELY CAUSE Prompt or target reports every tick	CHECK HEARTBEAT.md and delivery target	RECOVERY Require a condition and return no-op when healthy.

Source lens: aligned lab acceptance criteria



Security hardening

Architecture decision, operating boundary and acceptance evidence

Identity

Use pairing, allowlists, separate agent accounts and scoped service credentials.

Capability

Deny unnecessary runtime controls; separate read, draft and mutate tools.

Isolation

Sandbox untrusted inputs and test third-party skills without production access.

Recovery

Know how to stop schedules, revoke keys, preserve logs and restore trusted config.

Security hardening: core commands and configuration

```
{
  plugins: { allow: ["approved-plugin"] },
  tools: {
    deny: ["gateway", "cron", "sessions_spawn", "sessions_send"]
  },
  agents: {
    defaults: {
      sandbox: { mode: "non-main", scope: "agent" }
    }
  }
}

openclaw doctor
openclaw sandbox explain --json
```

1

plugins

Allow only reviewed in-process code.

2

deny

Remove escalation surfaces from untrusted agents.

3

sandbox

Isolate non-main sessions.

4

diagnose

Capture effective controls as evidence.

EXPECTED

Security checklist, sandbox explanation and channel policies match the design.

IF NOT

Treat any broad exception as a documented risk decision, not a quick fix.

Source lens: official OpenClaw docs plus the aligned learner lab

Security hardening: diagnose before changing

1	SYMPTOM unexpected tool	LIKELY CAUSE Policy inheritance or plugin surface	CHECK effective tool list and plugin allowlist	RECOVERY Deny or unload the capability at the correct scope.
2	SYMPTOM secret in output	LIKELY CAUSE Prompt/log leakage	CHECK transcript, logs and environment scope	RECOVERY Revoke key, redact evidence and redesign injection.
3	SYMPTOM prompt injection	LIKELY CAUSE Untrusted content treated as instruction	CHECK source, tool call and approval trace	RECOVERY Separate data, constrain tools and require approval.

Source lens: aligned lab acceptance criteria



Dashboard and observability

Architecture decision, operating boundary and acceptance evidence

Control UI

Inspect configuration, sessions and operator surfaces on the Gateway port.

Health

Differentiate process alive, provider reachable, channel healthy and workflow correct.

Logs

Correlate run ID, session key, tool calls, errors and delivery state.

Acceptance

Explain one successful run and one failure from dashboard plus logs.

Dashboard and observability: core commands and configuration

```
openclaw gateway status
openclaw health
openclaw logs
openclaw dashboard

# VPS: authenticated tunnel from local machine
ssh -L 18789:127.0.0.1:18789 user@server

# then open locally
http://127.0.0.1:18789
```

1

status

Confirm service and bind details.

2

health

Check operational dependencies.

3

logs

Follow the failing run boundary.

4

tunnel

Keep the UI private while accessing a VPS.

EXPECTED

Local browser reaches the remote loopback UI through the SSH tunnel.

IF NOT

Check the local port, SSH session and remote loopback bind separately.

Source lens: official OpenClaw docs plus the aligned learner lab



Dashboard and observability: diagnose before changing

1	SYMPTOM port in use	LIKELY CAUSE Local 18789 already occupied	CHECK Isof or alternate local port	RECOVERY Map 18790 locally to remote 18789.
2	SYMPTOM blank dashboard	LIKELY CAUSE Gateway unhealthy or UI disabled	CHECK health, logs and controlUi config	RECOVERY Repair the Gateway/UI configuration.
3	SYMPTOM missing run	LIKELY CAUSE Wrong session or filtered logs	CHECK run ID, agent ID and time range	RECOVERY Search the exact lifecycle identifiers.

Source lens: aligned lab acceptance criteria



Cost and context efficiency

Architecture decision, operating boundary and acceptance evidence

Context size

Bootstrap files, tool definitions, history and tool results consume input tokens.

Frequency

Cron, heartbeat and retries multiply recurring costs even when individual calls are cheap.

Model fit

Use a strong orchestrator where tool reliability matters and cheaper workers for bounded transforms.

Acceptance

Compare cost per verified outcome before and after routing or context changes.

Cost and context efficiency: core commands and configuration

```
{
  agents: {
    defaults: {
      model: { primary: "openai/gpt-5.4" },
      compaction: {
        notifyUser: true,
        memoryFlush: { model: "ollama/qwen3:8b" }
      }
    },
    entries: {
      summarizer: { model: "provider/low-cost-model" }
    }
  }
}

/status
/compact Focus on architecture decisions
```

1

orchestrator

Keep reliability where planning is complex.

2

worker

Route bounded summaries to lower cost.

3

flush

Use an exact housekeeping model.

4

compact

Preserve relevant decisions and recent tail.

EXPECTED

Lower repeat cost without a failed acceptance test or missing evidence.

IF NOT

A cheaper answer that requires rework is not an efficiency gain.

Source lens: official OpenClaw docs plus the aligned learner lab

Cost and context efficiency: diagnose before changing

1	SYMPTOM cost spikes	LIKELY CAUSE Fan-out, schedule or retry loop	CHECK usage by agent and trigger	RECOVERY Bound parallelism, frequency and retry count.
2	SYMPTOM context overflows	LIKELY CAUSE Large history or tool output	CHECK /status and compaction logs	RECOVERY Prune tool results, compact or start a task session.
3	SYMPTOM quality drops	LIKELY CAUSE Worker model below task requirement	CHECK acceptance failures and human corrections	RECOVERY Raise the route only for that role or condition.

Source lens: aligned lab acceptance criteria

Blockchain invoice verification

Architecture decision, operating boundary and acceptance evidence

External service

A small API registers and verifies invoice data against a trusted record.

Agent role

Collect approved fields, call the API, interpret status and report evidence.

Trust boundary

The agent does not own signing authority or silently correct mismatched data.

Acceptance

Original invoice verifies; tampered amount or reference fails with an auditable response.

Blockchain invoice verification: core commands and configuration

```
# start the learner API
docker build -t invoice-verify labs/lab-10-blockchain-invoice
docker run --rm -p 5000:5000 invoice-verify

# register a trusted invoice
curl -X POST http://127.0.0.1:5000/register \
-H "Content-Type: application/json" \
-d '{"invoice_id":"INV-2026-001","amount":1200}'

# verify the same payload
curl -X POST http://127.0.0.1:5000/verify \
-H "Content-Type: application/json" \
-d '{"invoice_id":"INV-2026-001","amount":1200}'
```

1

container

Isolate the learner verification service.

2

register

Create the trusted reference record.

3

verify

Compare the submitted payload.

4

tamper test

Change amount and require a failed result.

EXPECTED

Original payload verifies; altered invoice produces a negative result.

IF NOT

Distinguish API unavailable, invalid JSON, unknown invoice and mismatch.

Source lens: official OpenClaw docs plus the aligned learner lab

Blockchain invoice verification: diagnose before changing

1	SYMPTOM connection refused	LIKELY CAUSE Container or port is not available	CHECK docker ps and port mapping	RECOVERY Start the service and confirm localhost:5000.
2	SYMPTOM 400 response	LIKELY CAUSE Invalid JSON or missing field	CHECK response body and request payload	RECOVERY Correct schema without changing the trusted record.
3	SYMPTOM verification false	LIKELY CAUSE Payload differs from registered data	CHECK invoice ID and canonical fields	RECOVERY Report the mismatch; do not overwrite it automatically.

Source lens: aligned lab acceptance criteria

Read the system from outside in

	SYMPTOM No message	LIKELY CAUSE Channel, sender or Gateway ingress	CHECK account health, policy, bind and log	RECOVERY Restore the input path before testing the model.
	SYMPTOM No action	LIKELY CAUSE Tool invisible, denied or unapproved	CHECK effective tool set, sandbox and approval	RECOVERY Correct the narrow gate; do not grant all tools.
	SYMPTOM Wrong state	LIKELY CAUSE Session route, stale context or memory	CHECK session key, transcript and retrieved files	RECOVERY Fix isolation or start a clean task boundary.
	SYMPTOM No delivery	LIKELY CAUSE Destination, permission or channel outage	CHECK delivery event and receipt	RECOVERY Retry idempotently after the destination is healthy.

Source lens: official troubleshooting and the ten lab acceptance checks



Specify eight fields before granting autonomy

Field	Required decision	Evidence before go-live
Owner + outcome	Named accountable owner and measurable result	Acceptance statement signed off
Trigger + session	Event source, schedule, binding and session scope	Test event resolves the expected sessionKey
Context + state	Approved sources, workspace files, memory and retention	Context manifest and state location
Tools + data	Exact read, draft, write and runtime-control capabilities	Effective tool list and data-scope test
Approval gate	Consequential actions that must pause	Recorded approve/deny test
Evidence	Run ID, inputs, decisions, tool results and receipt	One complete evidence bundle
Failure mode	Timeout, retry, idempotency, escalation and stop	Controlled failure demonstration
SLO + cost	Success, latency, intervention and spend thresholds	Pilot measurements against targets

Source lens: official agent-loop, security and observability guidance



Score the workflow before choosing autonomy

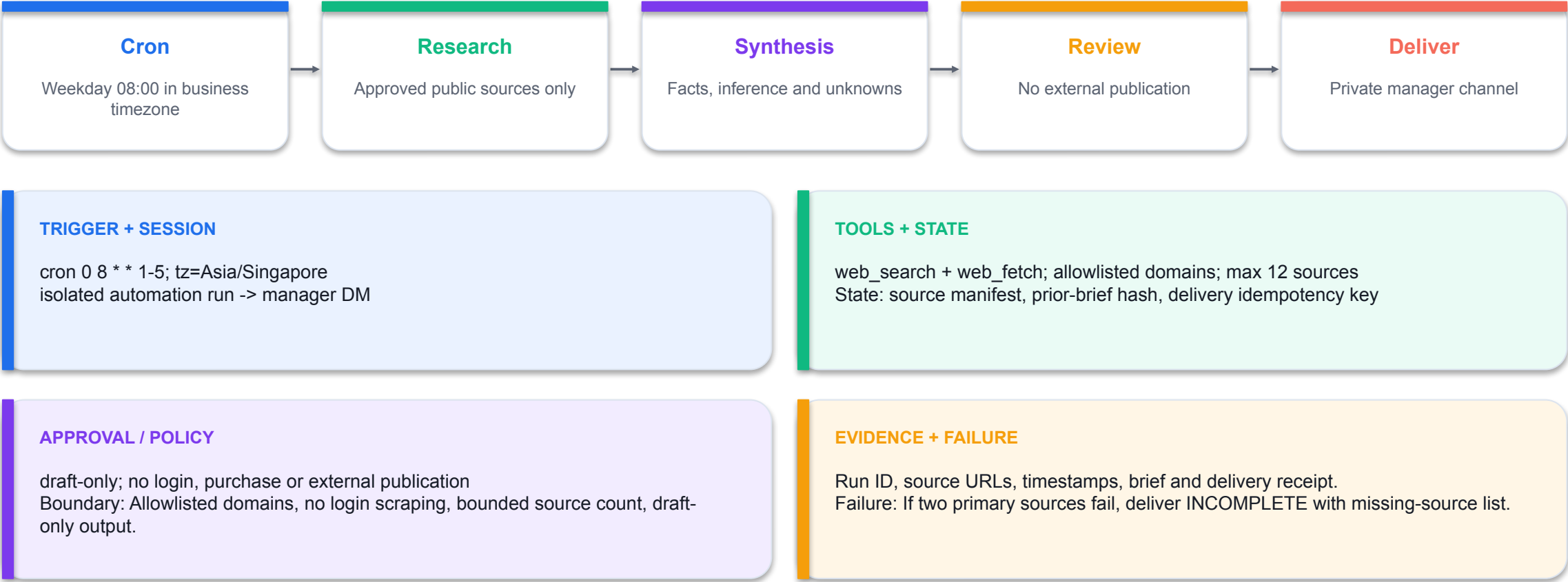
Candidate	Value	Reversibility	Data risk	Failure detectability	Decision
Daily source brief	5 - frequent	5 - draft only	4 - public data	5 - citations	Automate delivery
Ticket-response draft	4	5 - no send	3 - customer data	4 - human review	Automate draft
Refund execution	4	1 - money moves	2 - account data	3 - audit later	Require approval
Production deploy	5	2 - rollback cost	3 - code/secrets	2 - delayed signal	Keep human gate
Rule	Value x frequency	Higher is safer	Higher is safer	Higher is safer	Autonomy only when risk is bounded

Source lens: worked scoring model derived from the supplied use cases and security guidance



Daily executive research brief

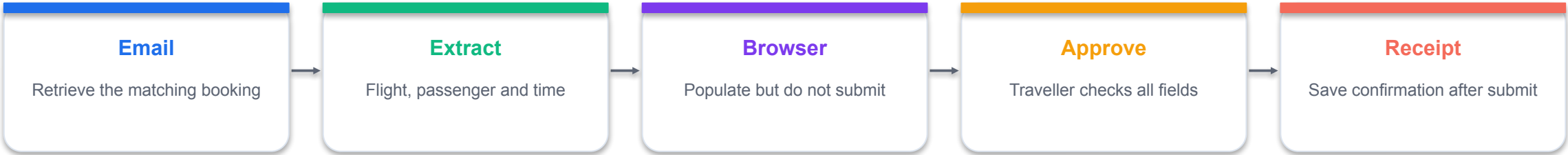
A manager wants a concise, source-linked market update at 08:00 every weekday.





Travel check-in assistant

The agent finds a booking email and prepares a browser check-in for the traveller.



TRIGGER + SESSION

approved booking-email match; one run per booking ID
private per-user session; expire after travel date

TOOLS + STATE

mail.read + browser.fill; browser.submit is approval-only
State: booking reference, extracted fields and preview screenshot

APPROVAL / POLICY

Traveller verifies name, flight, seats and passport fields before submit.
Boundary: Read-only email access; browser submission requires per-action approval.

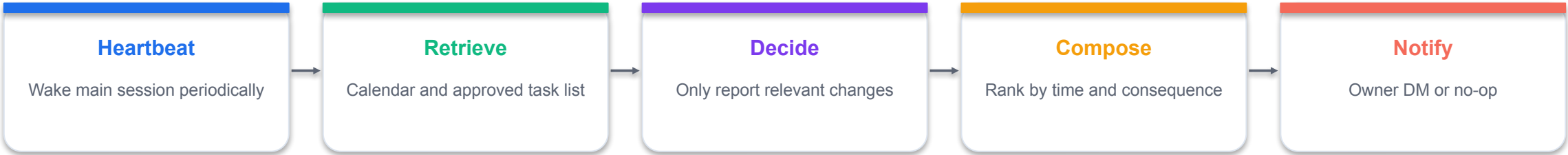
EVIDENCE + FAILURE

Booking source, preview screenshot, approval event and confirmation ID.
Failure: Mismatch or CAPTCHA stops the run and returns the populated preview.



Personal morning command centre

A user asks one channel for calendar, priority, weather and outstanding follow-ups.



TRIGGER + SESSION

heartbeat every 30m in the owner main session
owner-only main session; no detached job transcript

TOOLS + STATE

calendar.read, tasks.read, weather.read; no write capability
State: last-notified item hashes suppress duplicate alerts

APPROVAL / POLICY

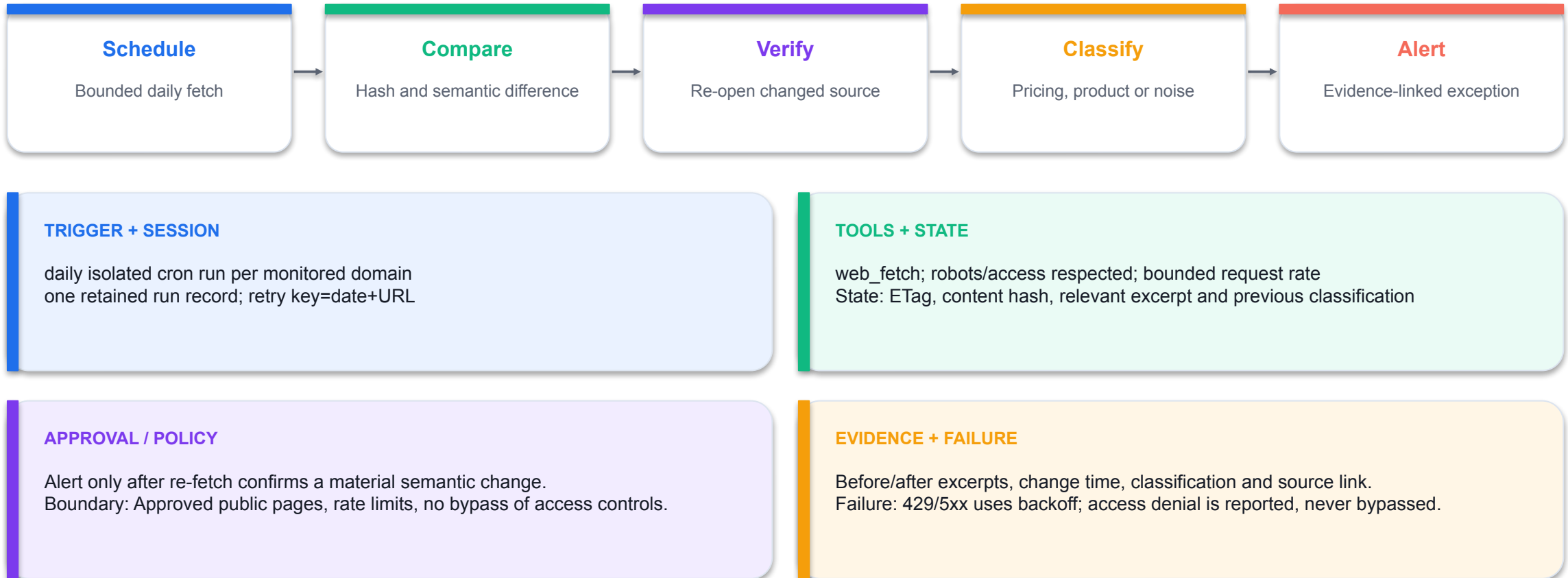
Notify only for a new deadline, conflict or weather exception.
Boundary: Owner-only target; read-only sources; no notification when nothing changed.

EVIDENCE + FAILURE

Retrieved items, prioritisation rationale and delivered message or no-op trace.
Failure: Unavailable source is labelled stale; healthy state returns no-op.

Competitor change monitor

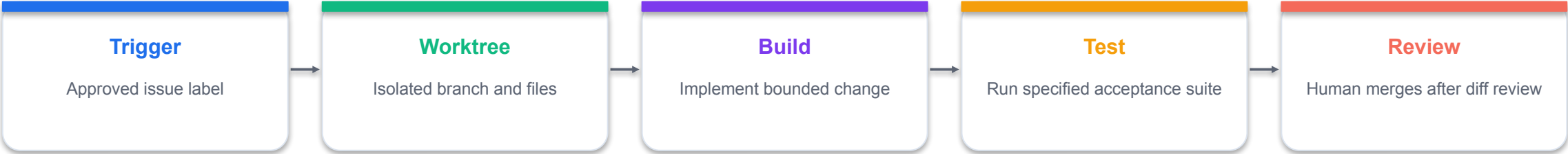
Operations needs to know when selected competitor pages materially change.





Repository issue-to-fix workflow

A development team wants an agent to investigate a labelled issue and prepare a reviewed patch.



TRIGGER + SESSION
repository issue receives the approved agent-ready label
isolated run + dedicated worktree/branch

TOOLS + STATE
repo read/write and named test commands; no deploy or main push
State: issue ID, branch SHA, diff, test log and reviewer comments

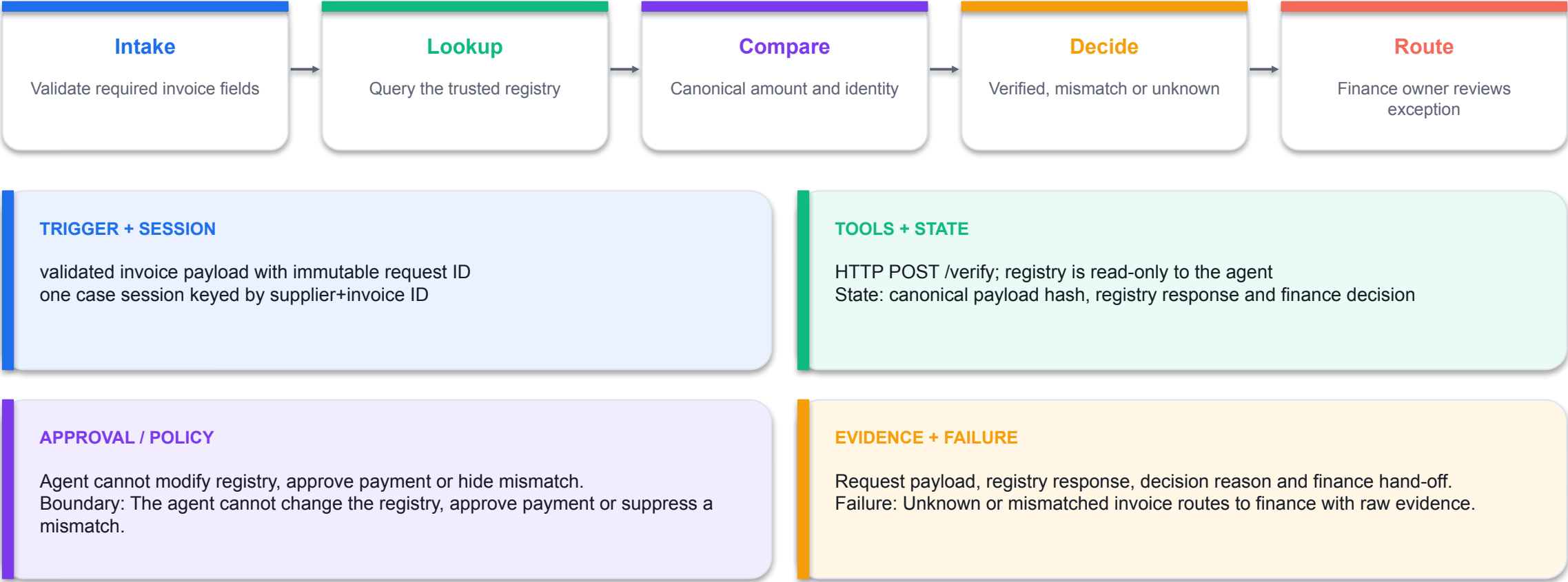
APPROVAL / POLICY
Human reviews diff and tests before merge authority is exercised.
Boundary: No direct main-branch write, no secret access, no deployment authority.

EVIDENCE + FAILURE
Issue ID, branch, diff, tests, reviewer decision and final commit.
Failure: Failing tests return the branch, diff and first failing assertion.



Supplier invoice verification

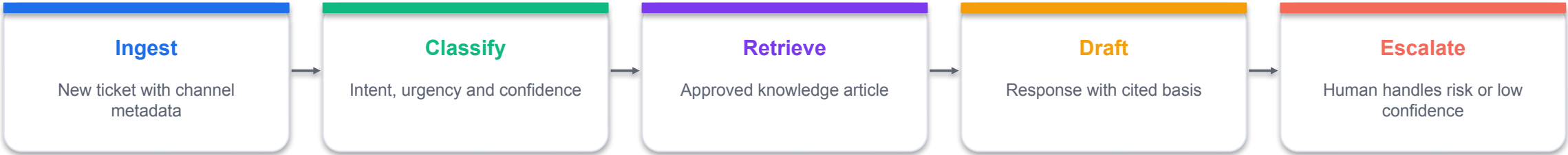
Finance needs a repeatable check against a trusted invoice registry before approval.





Customer-service triage

A shared inbox needs urgency, intent and knowledge-backed draft responses.



TRIGGER + SESSION

new shared-inbox ticket; bind session to ticket thread
per-thread session; customer identity never crosses tickets

TOOLS + STATE

mail.read, KB.retrieve and draft.create; refunds are denied
State: ticket ID, intent, confidence, article IDs and draft revision

APPROVAL / POLICY

Confidence <0.80, safety, refund or account change escalates.
Boundary: No autonomous refunds, account changes or policy exceptions.

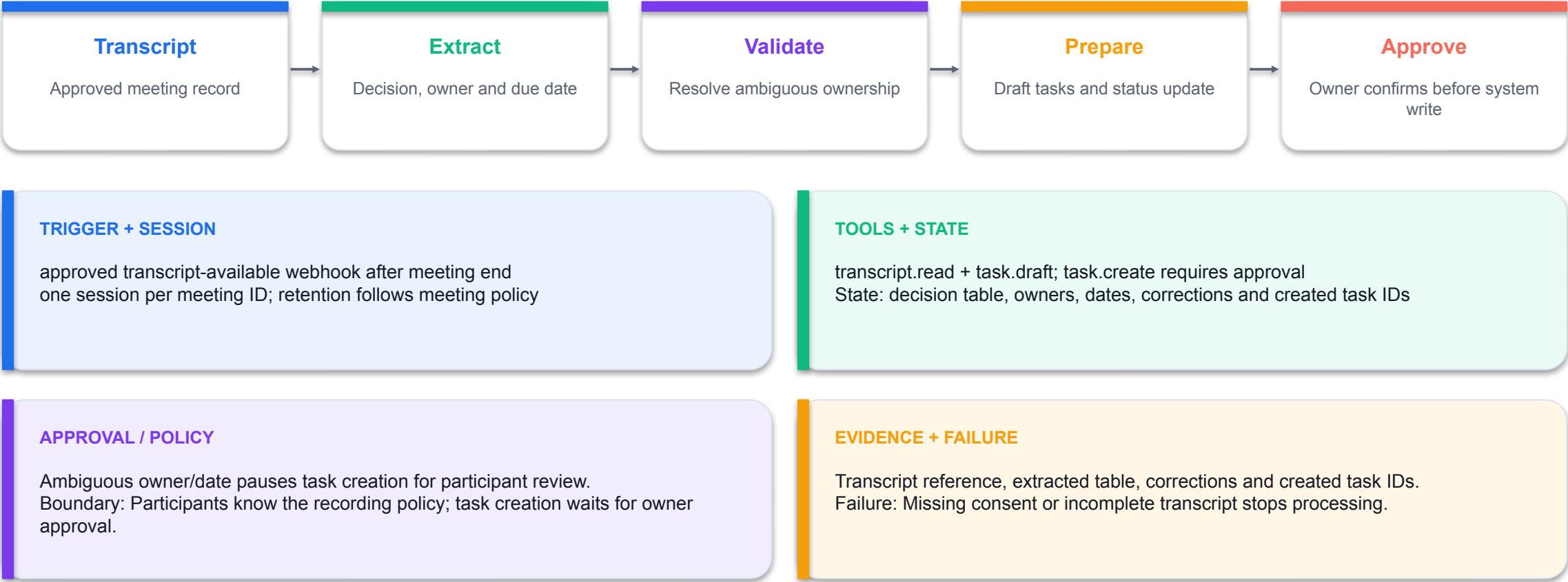
EVIDENCE + FAILURE

Ticket ID, classification, cited article, draft and escalation owner.
Failure: No grounded article produces a clarification, not an invented answer.



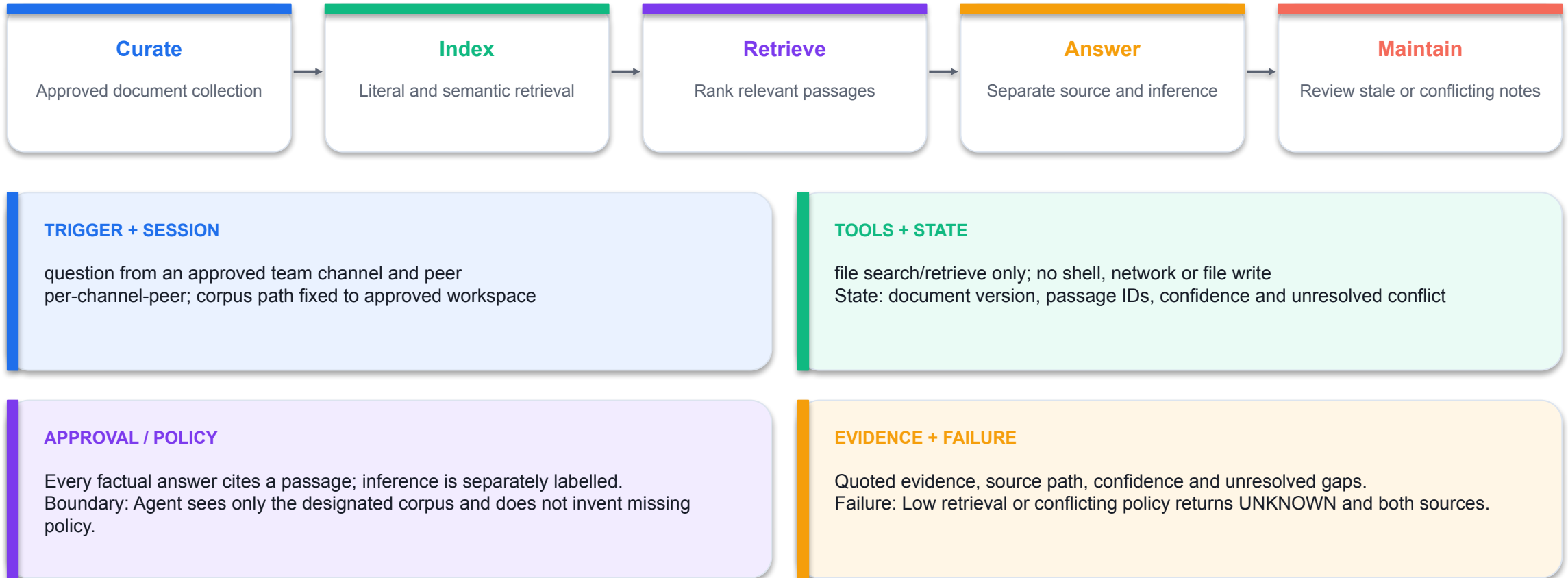
Meeting-to-action workflow

A project team wants decisions and assigned actions captured after each meeting.



Local knowledge assistant

A technical team asks questions over controlled procedures and project notes.





Multi-agent overnight builder

A product owner assigns one bounded prototype for morning review.





Bind each action class to an enforceable control

Action class	Examples	Default autonomy	Enforcement	Required evidence
Read / retrieve	Search, fetch, file read	Automatic inside scope	Data and network allowlist	Source, query, timestamp, result hash
Draft	Reply, report, task proposal	Automatic; no external effect	No send/write capability	Draft ID, citations, confidence
Reversible write	Label, create task, update staging	Bounded automatic	Idempotency key + rollback	Before/after, receipt, rollback test
Consequential write	Send message, refund, delete, deploy	Per-action approval	Approval gate + exact arguments	Approver, decision, tool result
Runtime control	Gateway, plugins, secrets, agent spawning	Denied to task agents	Separate operator identity	Change ticket and operator audit

Source lens: official tool-policy, sandbox and security documentation

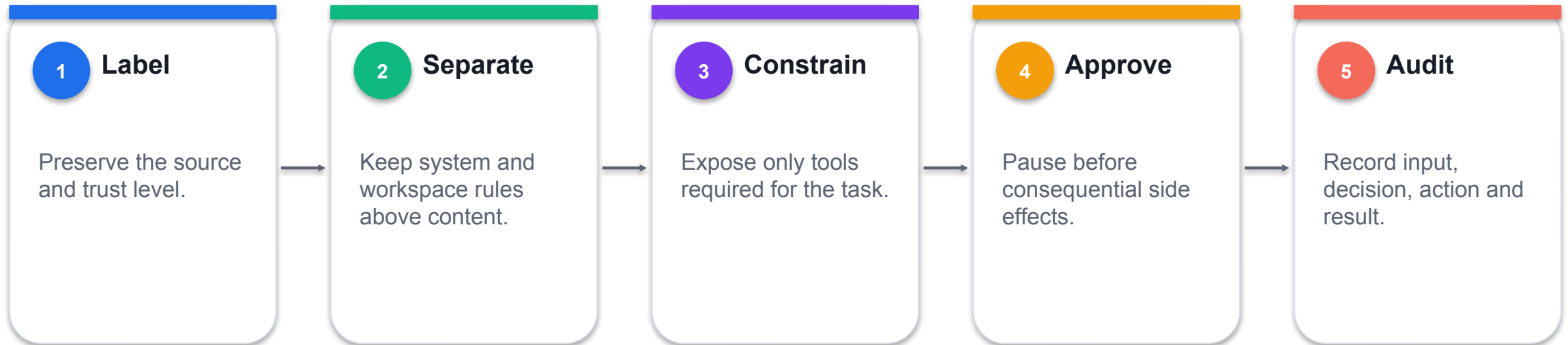


Model compromise must not become runtime compromise

Boundary	Attack	Primary control	Detection evidence
Channel ingress	untrusted sender / room instruction	pairing, allowFrom, mention gate, isolated dmScope	auth rejection and route decision
Retrieved content	prompt injection in page/file	data/instruction separation + source trust label	tainted-source annotation
Tool call	argument injection or destination pivot	schema validation, allowlist, approval, idempotency	args hash + policy decision
Sandbox/host	escape or overbroad filesystem/network	non-main sandbox, workspaceAccess none, host boundary	sandbox explain + denied syscall/action
Plugin/MCP	supply-chain or hidden capability	pinned reviewed package + plugin allowlist	manifest/hash and effective tool diff
Secrets	prompt/log/transcript exfiltration	SecretRef/runtime injection + redaction	secret access event without value

Source lens: official security, sandbox and skill/plugin trust guidance

Treat retrieved content as data, never authority



SYSTEM PATH Label -> Separate -> Constrain -> Approve -> Audit

Source lens: official security guidance



Bind authority to agent, session, tool and exact argument scope

Scope	Enforcement point	Safe default	Proof
Agent identity	binding -> agents.entries[]	separate specialist agent	resolved agentId and workspace
Session	dmScope/groupScope/thread binding	per-channel-peer for multi-user DMs	effective sessionKey
Tool	global/profile/agent allow-deny	deny runtime controls to task agents	effective tool list
Execution	sandbox mode/scope/workspaceAccess	non-main + session + none	sandbox explain --json
Arguments	command/destination allowlist + approval	ask on miss; deny unknown target	policy match + approver
Credential	SecretRef/service account	one purpose and minimal API scope	secret reference and provider audit

Source lens: official security, session, sandbox and tool-policy reference



Credential references and runtime snapshots prevent prompt leakage

Phase	Representation	Runtime behavior	Failure handling
Configuration	SecretRef or environment-backed reference	Config stores a reference, not the secret value	Schema/ref validation fails closed
Resolution	provider-specific secret lookup	Resolve only for the component requiring it	Missing secret blocks account/provider start
Snapshot	active in-memory secret snapshot	Successful reload atomically replaces active values	Failed reload keeps last good snapshot
Injection	narrow process/file/credential path	Secret is not placed in model prompt or skill text	Limit scope and lifetime
Logging	redacted metadata and safe reason codes	Record source/path/status without value	Scan logs/transcripts for accidental value
Rotation	new version + secrets reload	Revoke old credential after dependent surfaces pass	Rollback to last known-good reference

Source lens: official secrets management and Gateway reload reference



Define reliability with computable formulas and event sources

SLO	Formula	Example target	Required events
End-to-end success	delivered_once / accepted_runs	>= 99% over 7 days	accepted, terminal, delivery receipt
p95 latency	p95(deliveredAt - acceptedAt)	< 30 s interactive	acceptedAt, startedAt, model/tool spans, deliveredAt
Queue wait	p95(startedAt - acceptedAt)	< 2 s	queue accepted/start timestamps
Tool reliability	successful_calls / attempted_calls	>= 99.5% per tool	toolCallId, status, duration, retry
Human intervention	approved_or_corrected / completed_runs	< 10% for low-risk flow	approval and correction events
Cost per verified outcome	model+tool cost / accepted outcomes	workflow-specific budget	usage, tool charges, acceptance result

Source lens: official telemetry guidance and service-operation practice

Recover the control plane without destroying evidence

```
openclaw gateway status --deep
openclaw gateway status --require-rpc
openclaw channels status --probe
openclaw logs --follow
openclaw doctor
openclaw doctor --fix
openclaw secrets reload
openclaw gateway restart
```

```
# invalid configuration exits 78
# systemd: RestartPreventExitStatus=78
```

1

Ownership

--deep finds competing launchd/systemd/schtasks services.

2

RPC proof

--require-rpc distinguishes process reachability from usable control plane.

3

Safe mode

Repeated unclean boots suppress channel/provider auto-start for inspection.

4

Exit 78

Invalid config must stop restart storms until repaired.

EXPECTED

One lifecycle owner, valid config, authenticated RPC, healthy channels and preserved transcripts.

IF NOT

Do not delete state or reinstall before exporting logs, config path, session identifiers and the first failing timestamp.

Source lens: official Gateway runbook, restart recovery and Doctor reference

Submit an implementation manifest that an operator can enforce

```
workflow:
  owner: finance-ops
  trigger: {type: webhook, idempotency: invoice_id}
  route: {agent: invoice, session: per-case}
  data: {allow: [invoice_registry], retention: 30d}
  tools: {allow: [http_verify], deny: [exec, gateway, cron]}
  approval: {required_for: [payment_release]}
  failure: {timeout_s: 20, retries: 2, escalate: finance}
  evidence: [run_id, input_hash, tool_result, approver, receipt]
  slo: {success: 0.99, p95_s: 30, max_cost_sgd: 0.25}
```

1

Deterministic route

Trigger, agent and session scope are explicit.

2

Least privilege

Allowed data/tools and denied runtime controls are machine-reviewable.

3

Failure contract

Timeout, retry and escalation are bounded before go-live.

4

Acceptance

Evidence fields and SLOs turn a demo into an operable service.

EXPECTED

A reviewer can map every manifest field to configuration, runtime evidence and an acceptance test.

IF NOT

Reject the design if any consequential action lacks an approval gate, receipt or rollback/escalation path.

Source lens: derived implementation contract using official security, routing and telemetry guidance

Official product, architecture and operations

Reviewed 23 August 2026 - part 1 of 3

- 1 **Official documentation** <https://docs.openclaw.ai/>
- 2 **Official FAQ** <https://docs.openclaw.ai/help/faq>
- 3 **Official getting started** <https://docs.openclaw.ai/start/getting-started>
- 4 **Official showcase** <https://openclaw.ai/showcase>
- 5 **OpenClaw repository** <https://github.com/openclaw/openclaw>
- 6 **OpenClaw GitHub organisation** <https://github.com/openclaw>
- 7 **OpenClaw Foundation** <https://www.openclaw.org/>

Official documentation governs technical facts; independent sources broaden examples and risk discussion.

Use cases and deployment perspectives

Reviewed 23 August 2026 - part 2 of 3

- 1 **Android app listing** https://play.google.com/store/apps/details?id=ai.openclaw.app&hl=en_US
- 2 **Awesome OpenClaw use cases** <https://github.com/hesamsheikh/awesome-openclaw-usecases>
- 3 **Awesome OpenClaw directory** <https://github.com/rohitg00/awesome-openclaw/blob/main/README.md>
- 4 **Hostinger use-case guide** <https://www.hostinger.com/tutorials/openclaw-use-cases/>
- 5 **AI Agents Kit use-case guide** <https://aiagentskit.com/blog/openclaw-use-cases/>
- 6 **Kanerika use-case guide** <https://kanerika.com/blogs/openclaw-usecases/>
- 7 **DigitalOcean overview** <https://www.digitalocean.com/resources/articles/what-is-openclaw>

Official documentation governs technical facts; independent sources broaden examples and risk discussion.

Safety, ecosystem and additional perspectives

Reviewed 23 August 2026 - part 3 of 3

- 1 TechRadar overview** <https://www.techradar.com/pro/what-is-openclaw>
- 2 Malwarebytes safety analysis** <https://www.malwarebytes.com/blog/news/2026/02/openclaw-what-is-it-and-can-you-use-it-safely>
- 3 IONOS overview** <https://www.ionos.com/digitalguide/server/know-how/openclaw/>
- 4 Milvus architecture guide** <https://milvus.io/blog/openclaw-complete-guide>
- 5 Tencent Cloud TechPedia** <https://www.tencentcloud.com/techpedia/140975>
- 6 Emergent guide** <https://emergent.sh/learn/what-is-openclaw>
- 7 Wikipedia overview** <https://en.wikipedia.org/wiki/OpenClaw>

Official documentation governs technical facts; independent sources broaden examples and risk discussion.

Use the LMS for the current learner materials

Download the learner slide PDF, Learner Guide, labs and learner-facing assessment papers

<https://lms-tms.tertiaryinfotech.com/>

1 Download

Use the current v6.0 slides and guide.

2 Complete labs

Retain the required evidence.

3 Take assessment

Open learner papers only.

4 Submit

Follow the course record instructions.

Answer keys remain trainer-only and are never attached to the learner course record.

Final reminder before you begin

30 min WA

Answer the architecture and operating-control questions in your own words.

30 min Case Study

Apply the ten labs to the stated business scenario.

Open book

Use the current slides, Learner Guide and approved lab materials.

Attendance

Complete the Assessment digital attendance first.

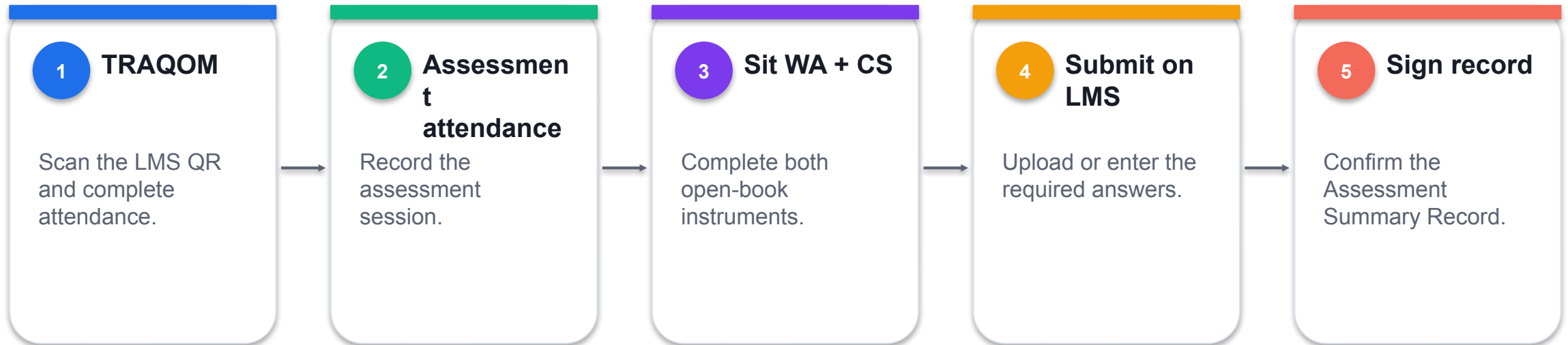
Submission

Submit both learner-facing instruments through the LMS.

Record

Sign the Assessment Summary Record after completion.

Complete every assessment step in order



SYSTEM PATH TRAQOM -> Assessment attendance -> Sit WA + CS -> Submit on LMS -> Sign record

Digital Attendance (Mandatory)

Complete this immediately before the assessment begins

Scan

Use the official assessment attendance QR code shown by the trainer.

Verify

Confirm your identity, course code and assessment session.

Retain

Keep the confirmation screen until the trainer verifies completion.

THANK YOU

**Understand the system.
Bound the authority.
Prove the result.**

Questions - reflections - assessment

